SC3020 Data Analytics & Mining
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Introduction to Data Analytics and Mining

"Data is the new oil, but analytics is the refinery." Mining turns raw records into decisions. This chapter builds the why and what of the field from everyday examples to a precise process model.

From zero: no analytics background assumed. We start from a shopkeeper's ledger, define *data, information, knowledge*, introduce types of data you will meet in every project, then formalise the CRISP-DM lifecycle. Pictures precede formalism.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish data, information, and knowledge and explain the analytics pipeline;

(L2) classify data types (nominal, ordinal, interval, ratio; structured vs. unstructured);

(L3) describe the six phases of CRISP-DM and their iterative nature;

(L4) identify ethical issues (privacy, bias, fairness) and relevant regulations;

(L5) scope a small analytics project using a CRISP-DM checklist.

1.1 Why Analytics? From Ledger to Decisions

Intuition. A neighbourhood bakery writes daily sales in a notebook: time, item, price, customer. The notebook is *data* (raw facts). Counting "croissants sell twice as fast on Saturdays" is *information* (processed data). Deciding "bake 30 extra croissants Saturday morning" is *knowledge* (actionable information). *Analytics* is the refinery between them; *data mining* is the set of algorithms that find the non-obvious patterns.

Formal definitions. Let a *dataset* be a collection $\mathcal{D} = \{x_i\}_{i=1}^{n}$ where each record $x_i \in \mathcal{X}$ describes an entity via attributes (features). An *analytics pipeline* transforms $\mathcal{D}$ into a model M and then into a decision d that

maximises expected utility under a business objective. Two classic frameworks organise this: *KDD* (Knowledge Discovery in Databases, Fayyad et al.) and *CRISP-DM* (Cross-Industry Standard Process for Data Mining). KDD is academic (selection → preprocessing → transformation → mining → interpretation); CRISP-DM is industrial and we adopt it here.

1.2 Types of Data

Intuition. Not all columns behave alike. You can average a temperature but not a postcode, and you can rank "small < medium < large" but not compute "medium minus small." Using the wrong operation is a silent bug.

Formal taxonomy. Stevens' scales:

Nominal Categories without order (e.g., blood type, country). Only $=$, $\neq$, mode.

Ordinal Ordered categories without meaningful gaps (e.g., rating 1–5, education level). Adds $<$ and median.

Interval Numeric with arbitrary zero (e.g., Celsius, calendar year). Differences meaningful, ratios not (20° is not "twice as hot" as 10°).

Ratio Numeric with absolute zero (e.g., age, price, Kelvin). All arithmetic valid.

Orthogonally, *structured* data sits in tables (rows = records, columns = attributes); *semi-structured* has self-describing tags (JSON, XML); *unstructured* has no fixed schema (text, images, audio). Most mining value now lies in the latter two, but almost always begins by *structuring* them (e.g., bag-of-words).

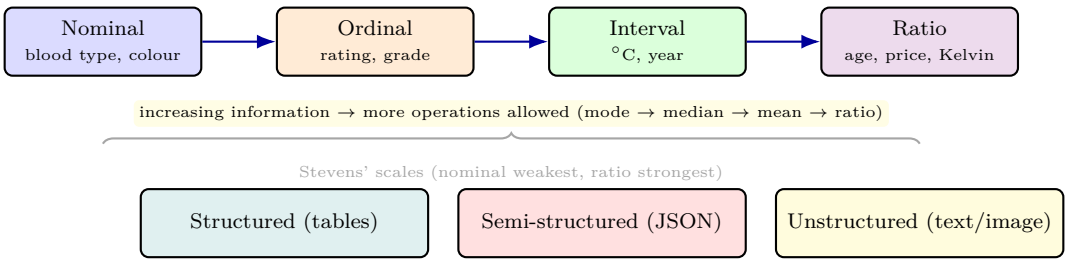

Figure 1.1: Data type hierarchy. Higher scales inherit operations of lower ones. Structured vs. semi/unstructured is an orthogonal axis about schema regularity.

1.3 CRISP-DM: The Mining Lifecycle

Intuition. Without a process, projects wander: a brilliant model that solves the wrong question, or a perfect question with no deployable answer. CRISP-DM prevents that by forcing you to define success *before* touching data.

Six phases. (1) **Business Understanding**: translate a business goal ("reduce churn 10%") into a data-mining goal ("predict P(churn | behaviour)"). Deliverable: success criteria. (2) **Data Understanding**: collect,

describe, explore, verify quality. (3) **Data Preparation** (typically 60–80% of effort): cleaning, integration, transformation, reduction (Ch. 2). (4) **Modeling**: select and calibrate algorithms (Ch. 3–6). (5) **Evaluation**: test against business criteria, not just accuracy (Ch. 7). (6) **Deployment**: integrate, monitor, maintain. Crucially the process is *cyclic*: failure at Evaluation loops back to Business Understanding; insights in Modeling reveal new data needs.

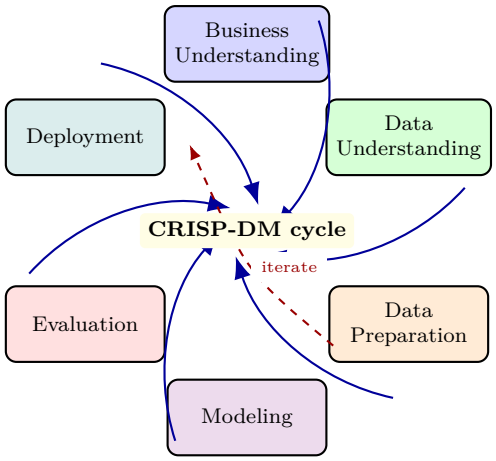

Figure 1.2: CRISP-DM cycle. The outer ring is the nominal sequence; inner feedback edges are the norm in practice, not the exception.

Listing 1.1: Classifying columns before modelling avoids illegal operations.

```
import pandas as pd
df = pd.DataFrame({"colour":["red","blue","red"], "rating":[1,3,5],
                   "temp_C":[10,20,30], "price":[3.5,4.0,5.5]})

# Stevens checks: what is meaningful?
print(df["colour"].mode())          # OK for nominal
print(df["rating"].median())        # OK for ordinal+
print((df["temp_C"].mean(), df["price"].mean()))  # both numeric
# Forbidden: ratio on interval
# print("20C twice 10C?", 20/10)   # meaningless for Celsius

# Schema check
print(df.dtypes)                    # structured table already
# For JSON column: pd.json_normalize(df["json_col"])
```

1.4 Ethics, Privacy, and Governance

Intuition. The bakery knows who buys what and when. Publishing "customer X buys cake every Friday" may be harmless; linking it to health records is not. All analytics trades off insight against intrusion.

Framework. Three pillars: *privacy* (control over personal data — GDPR, PDPA Singapore, consent, anonymisation, k-anonymity), *bias and fairness* (data reflects history; models amplify it — e.g., hiring data biased against a group yields discriminatory predictions; mitigate via balanced sampling, fairness metrics, audit), and *account-*

ability (provenance, reproducibility, explainability, human-in-the-loop for high-stakes decisions). A useful test before deployment: would the affected person, told how the model uses their data, consider it fair? If not, revisit Business Understanding.

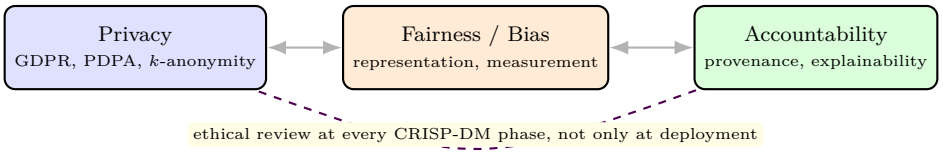

Figure 1.3: Ethics triangle. The three concerns interact; a fix for one can harm another (e.g., removing a sensitive attribute may hide bias rather than remove it).

1.5 Analytics versus Mining versus Science

Intuition. Vendors conflate terms. Think of a hierarchy: *data analytics* is the umbrella (any analysis of data for insight), *data mining* is the algorithmic core (pattern discovery at scale, often without a pre-stated hypothesis), and *data science* adds engineering and communication (pipelines, deployment, storytelling). A data analyst asks "what happened?", a miner asks "what pattern holds generally?", a scientist asks "how do we build a system that keeps answering?"

Example. Retail basket analysis: analytics reports "milk sales up 8%"; mining discovers "{diapers} $\Rightarrow$ {beer} on Fridays with lift 2.1"; data science builds a recommender that deploys the rule, A/B tests it, and monitors drift. The CRISP-DM phases map: analytics lives in Data Understanding, mining in Modeling, science spans the full cycle.

Remark. In NTU SC3020, "Data Analytics & Mining" emphasises both ends: you learn the statistical thinking of analytics and the algorithmic toolkit of mining, plus the engineering to make results reproducible.

1.6 Running Example: Telco Churn

We will revisit a churn problem throughout the book. Business goal: reduce churn 10%. Data: 7,043 customers $\times$ 21 attributes (tenure, contract, monthly charges, support calls). Preparation reveals missing tenure for 11 new customers and MonthlyCharges in SGD vs. USD (inconsistent). Modeling compares trees, forests, and boosting; Evaluation uses AUC and expected savings (not raw accuracy, because churn is 26% positive). Deployment triggers a retention campaign whose uplift is the true business metric.

1.7 Visualising the Pipeline

Scatter and histogram. The first exploratory plots are histograms (distribution of one feature), scatter matrices (pairs), and boxplots (median, quartiles, outliers). A histogram of income reveals skew; log-transform may symmetrise.

Correlation heatmap. Matrix $R_{ij} = \text{corr}(x_i, x_j)$ coloured from -1 (red) to $+1$ (blue) instantly shows redundancy: two highly correlated features carry the same signal — drop one or combine via PCA (Ch. 2). Colour is not decoration; it guides feature selection before any model is fit.

Interactive view. In practice, dashboards (Tableau, PowerBI, or matplotlib/plotly in Python) let stakeholders brush outliers and see business impact. This aligns CRISP-DM's Data Understanding with Business Understanding.

1.8 Mini-Case: Scoping with CRISP-DM

Scenario. An e-commerce site wants to reduce cart abandonment. Business Understanding: abandonment is 68%; goal is -10 points in 3 months. Success metric: conversion rate on treated sessions, not model AUC.

Data Understanding. Sources: clickstream (semi-structured JSON), transactions (structured), and support chats (unstructured). Quality check finds 12% of sessions have no user-agent, and checkout timestamps are in mixed timezones.

Preparation and modelling peek. Clickstream is sessionised; chats are tokenised and TF-IDF'd (Ch. 8); abandonment is labelled as no purchase within 30 minutes. This scoping ensures the later 60% effort on preparation serves a defined decision.

Ethics check. Clickstream + purchase history is personal data under GDPR/PDPA. Consent for analytics must be obtained, and abandonment interventions must not discriminate by protected attributes inferred from browsing.

Takeaway. A one-page charter covering goal, data inventory, metric, and ethics prevents the common failure of building a model nobody can deploy. Revisit the charter after each CRISP-DM loop.

From charter to deployment. The cart-abandonment charter is only the entry ticket to the CRISP-DM loop, not its conclusion. Deployment means wiring the abandonment score into the checkout page so an intervention triggers within seconds of hesitation. That wiring forces a conversation between modellers, engineers, and product owners that no offline notebook can substitute. Evaluation then shifts from conversion rate on a pilot to sustained lift measured by randomised exposure over several weeks. If lift fades, the team loops back to Business Understanding rather than tuning hyperparameters in isolation.

Why preparation dominates. Clickstream JSON arrives nested, timestamped in mixed zones, and joined against transactions by fragile session keys. Cleaning these joins, deduplicating events, and labelling abandonment consistently typically consumes the bulk of project effort. The lesson for big-data work is that volume amplifies messiness instead of washing it out. A million sessions with systematic timezone skew mislead more

confidently than a hundred clean ones. Sampling down to a manageable slice for exploration is therefore legitimate, provided the sampling preserves weekly seasonality.

Monitoring and maintenance. Once live, abandonment behaviour drifts with seasons, promotions, and site redesigns. The team should track input distributions, score distributions, and intervention uptake on a dashboard reviewed weekly. A sudden drop in user-agent coverage, for instance, signals an instrumentation change that invalidates the training data. When drift is confirmed, the CRISP-DM cycle restarts at Data Understanding with fresh labels rather than a blind refit.

Scaling up to big data. At web scale the sessionisation step moves from pandas scripts to distributed processing with partitioning by day. The modelling logic stays identical, but data quality checks must run as automated assertions inside the pipeline. This separation of concerns keeps the science stable while the engineering scales horizontally with traffic.

Practical takeaway. Treat the one-page charter as a living document updated after every loop through evaluation. Record what was tried, what moved the metric, and which ethical review each intervention passed. That record becomes the institutional memory that lets the next analyst start from understanding rather than from zero. Students replicating this mini-case can mimic the loop on any public clickstream sample with a spreadsheet. Define the goal, inventory the fields, fix one quality issue, and report a single decision-oriented metric. The habit of scoping before modelling transfers to every later chapter of this book.

1.9 Chapter Notes

CRISP-DM (Chapman et al., 1999) remains the de-facto industry standard; KDD (Fayyad et al., 1996) is its academic cousin. Stevens (1946) defined the four scales. For Singapore context, see PDPA (Personal Data Protection Act). Good practice: start every project with a one-page CRISP-DM charter — business goal, data inventory, success metric, ethical risks.

1.10 Exercises

E1.1 **Types.** Classify each variable and state which statistics are valid: (a) postal code, (b) star rating (1–5), (c) temperature in Kelvin, (d) time-stamp, (e) free-text review. Give one illegal operation for each.

E1.2 **CRISP-DM trace.** A telco wants to cut churn 10%. Map the goal through all six CRISP-DM phases, naming one deliverable and one risk per phase. Where would you loop back if Evaluation shows “high accuracy but no business lift”?

E1.3 **KDD vs. CRISP-DM.** Draw both cycles side-by-side and mark where Business Understanding and Deployment appear. Why does industry prefer CRISP-DM?

E1.4 **Ethics case.** A hospital model predicts readmission from historical admissions. The data under-represents one ethnic group. (a) Which bias type? (b) Propose a data-level and a model-level mitigation. (c) Which

GDPR principle is at stake if patients did not consent to secondary use?

E1.5 **Small project.** Choose a public dataset (e.g., Iris, Titanic). Write a one-page charter: business question, data description, success metric, ethical note, and a CRISP-DM plan with estimated effort per phase.

Chapter 2

Data Preprocessing

"Garbage in, garbage out." No algorithm rescues bad inputs. Preprocessing — cleaning, transforming, and reducing data — is where most projects succeed or fail.

From zero: no preprocessing assumed. We start from a messy spreadsheet with missing cells and strange outliers, explain why each defect hurts models, then formalise cleaning, normalisation, and reduction. Pictures precede equations.

Learning Outcomes

After this chapter you will be able to:

(L1) diagnose missingness mechanisms (MCAR/MAR/MNAR) and choose imputation strategies;

(L2) apply normalisation (min-max, z-score, robust) and explain when each fails;

(L3) handle outliers and noisy data with binning, clipping, and regression smoothing;

(L4) reduce dimensionality via feature selection, sampling, and PCA intuition;

(L5) build a reproducible preprocessing pipeline in Python.

2.1 Why Preprocessing Dominates

Intuition. Imagine training a loan model where 20% of incomes are blank, one entry reads "age = 231," and salary is in dollars while age is in years. Distance-based learners (kNN, k-means) will be dominated by salary; a single outlier can flip a mean; missing values crash most code. Real data is incomplete, noisy, and inconsistent because it comes from humans, sensors, and merged systems.

Quality dimensions. Accuracy (correct values), completeness (no missing), consistency (no contradictions), timeliness (up-to-date), and interpretability. A *preprocessing pipeline* chains operations $T_k \circ \cdots \circ T_1(\mathcal{D})$ that

map raw $\mathcal{D}$ to analysis-ready $\mathcal{D}'$ while preserving signal and documenting every choice for reproducibility.

2.2 Cleaning: Missing Values and Noise

Missingness. Mechanisms: *MCAR* (missing completely at random, e.g., sensor glitch independent of data), *MAR* (missing depends on observed data, e.g., higher incomes less likely to be reported), *MNAR* (missing depends on the missing value itself, e.g., people with very high debt hide it). Strategies: row/column deletion (only if MCAR and few), mean/median/mode imputation, regression or kNN imputation, and model-based (EM). Deletion under MAR/MNAR biases results.

Noise and outliers. *Binning* (equal-width, equal-frequency, then smooth by mean/median/boundary), *regression smoothing*, *clipping* (winsorisation), and *clustering*: points far from any cluster are candidate outliers. Rule-of-thumb: $|z| > 3$ flags a univariate outlier; multivariate needs Mahalanobis distance or isolation forest.

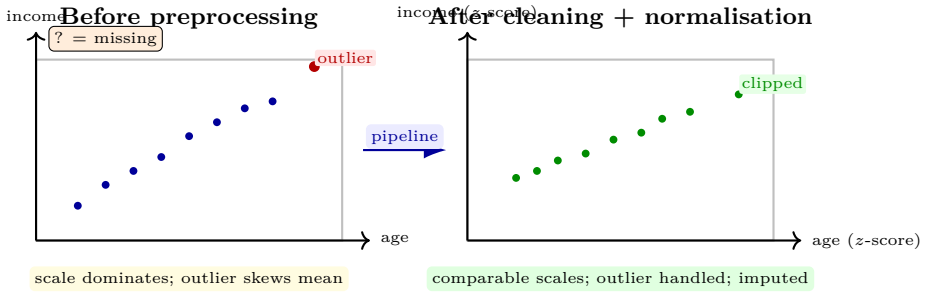

Figure 2.1: Preprocessing effect. Left: raw data with an outlier and missing marker; right: after imputation, clipping, and standardisation, both features contribute comparably.

2.3 Transformation: Normalisation and Discretisation

Intuition. If one feature ranges $[0, 10000]$ and another $[0, 1]$, Euclidean distance is just the first feature. Normalisation puts features on comparable footing; discretisation turns numbers into bins when algorithms need categories.

Formal methods. For feature x with min $x_{\min}$, max $x_{\max}$, mean μ, std σ:

$$\text{Min-max:} \quad x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1], \qquad \text{sensitive to outliers.} \tag{2.1}$$

$$\text{Z-score:} \quad x' = \frac{x - \mu}{\sigma} \quad (\mu_{x'} = 0,\ \sigma_{x'} = 1), \qquad \text{assumes roughly Gaussian.} \tag{2.2}$$

$$\text{Robust:} \quad x' = \frac{x - \text{median}}{\text{IQR}}, \quad \text{IQR} = Q_3 - Q_1, \qquad \text{outlier-resistant.} \tag{2.3}$$

Decimal scaling ($x/10^j$) and log transforms handle heavy tails. *Discretisation*: equal-width (cut range into k intervals), equal-frequency (each bin has n/k points), and entropy-based (choose cuts that maximise class separation). Normalise *after* splitting into train/test to avoid leakage.

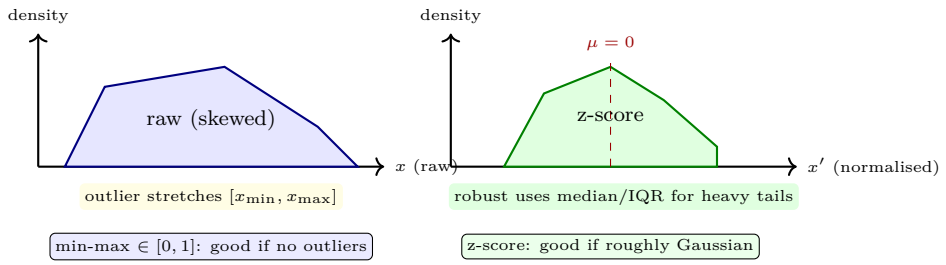

Figure 2.2: Normalisation choices. Min-max preserves distribution shape but an outlier compresses all other points; z-score centres at zero; robust scaling survives outliers.

Listing 2.1: A reproducible pipeline: impute, handle outliers, scale — fit on train only.

```python
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, RobustScaler

df = pd.DataFrame({"age":[25,30,np.nan,231,45], "income":[40,55,60,65,200]})
# Correct split BEFORE fitting transformers (avoid leakage)
train, test = train_test_split(df, test_size=0.3, random_state=0)

# 1. Impute (median resists the 231 outlier)
imp = SimpleImputer(strategy="median")
train_imp = imp.fit_transform(train)      # fit on train only
test_imp  = imp.transform(test)           # apply to test

# 2. Clip univariate outlier (|z|>3) -- simple winsorisation
def winsorise(X, k=3):
    mu, sd = X.mean(axis=0), X.std(axis=0, ddof=0)
    return np.clip(X, mu - k*sd, mu + k*sd)
train_clip = winsorise(train_imp)

# 3. Scale
scaler = StandardScaler()                 # or RobustScaler for heavy tails
train_scaled = scaler.fit_transform(train_clip)
test_scaled  = scaler.transform(test_imp)
print(train_scaled.mean(axis=0), train_scaled.std(axis=0))
```

2.4 Reduction: Selecting and Compressing

Intuition. Hundreds of features slow learners, add noise, and invite overfitting (curse of dimensionality). Reduction keeps signal, drops redundancy.

Families. *Feature selection*: filter (rank by correlation/mutual information), wrapper (search subsets, evaluate with model), embedded (LASSO, tree importance). *Feature extraction*: create new features, e.g., *PCA* (find orthogonal directions of maximal variance; project onto top k). Intuition: if data lie near a line in $\mathbb{R}^2$, PCA rotates axes so one coordinate captures almost all variance. *Sampling*: row reduction (random, stratified), and *data cube aggregation* (roll-up). *Numerosity*: replace data by parameters (regression) or histograms.

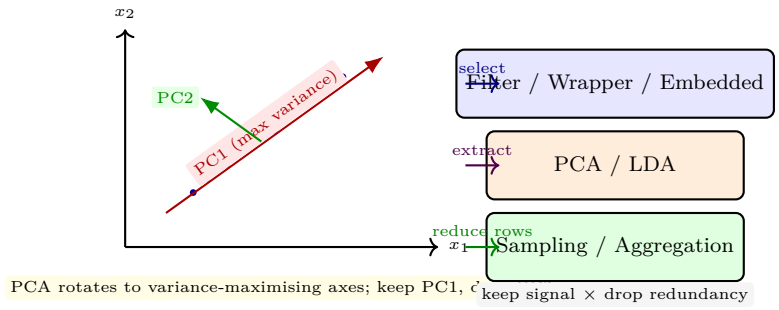

Figure 2.3: Left: PCA intuition on a 2D cloud. Right: reduction families. Selection keeps original features; extraction synthesises new ones.

2.5 Integration, Categorical Encoding, and Time

Entity resolution. Merging sources introduces duplicates ("Jon Tan" vs. "Tan Jon") and key mismatches. Techniques: deterministic rules, fuzzy matching (edit distance, Jaccard on tokens), and blocking to avoid $O(n^2)$ comparisons.

Categorical encoding. Nominal: one-hot (ℓ categories $\to \ell$ binary columns) or target encoding (replace by $P(y = 1 \mid \text{category})$, risky without regularisation). Ordinal: integer coding preserving order. High-cardinality: hashing trick or embeddings. Text dates: parse to components (year, month, day-of-week) rather than raw string.

Time issues. Timestamps require timezone normalisation, handling daylight saving, and creating derived features (tenure = today − start-date). For leakage prevention, compute tenure as of the observation date, not today.

Listing 2.2: Encoding categoricals and dates without leakage.

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder
df = pd.DataFrame({"colour":["red","blue","red"], "date":["2023-01-15","2023-02-20","
    2023-03-10"]})
enc = OneHotEncoder(sparse_output=False, handle_unknown="ignore")
print(enc.fit_transform(df[["colour"]]))
df["date"] = pd.to_datetime(df["date"])
df["month"] = df["date"].dt.month  # derived, not raw string
print(df)
```

2.6 Outlier Detection in Depth

Univariate vs. multivariate. A point can be normal on each axis yet outlying jointly (e.g., age 25 with income 200k). Mahalanobis distance $d_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})}$ accounts for covariance.

Isolation forest. Randomly partition space until a point is isolated; outliers isolate quickly (short path length). No distributional assumption and scales to high d.

Action. Options: remove (only if error), impute, cap (winsorise), or keep and use robust models (median, tree ensembles). Document the choice — an auditor must be able to reproduce your N.

2.7 Chapter Notes

Han, Kamber & Pei, Ch. 3 covers preprocessing comprehensively. Practical tip: log every transformer fitted on train and serialise the pipeline (e.g., `sklearn.pipeline.Pipeline`) so test and deployment data receive identical processing — otherwise you ship a training-serving skew bug.

2.8 Exercises

E2.1 **Missingness.** A survey column "income" is missing more often for high earners. Name the mechanism, explain why mean imputation biases the mean downward, and propose a better approach.

E2.2 **Normalisation.** Data $x = [0, 0.5, 1.0, 10]$. Compute min-max, z-score, and robust (median/IQR) for $x = 10$. Show an outlier compresses min-max and why robust resists.

E2.3 **Binning.** Smooth data $[4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34]$ by equal-frequency 3 bins using (a) bin means, (b) bin boundaries. Compare denoising vs. information loss.

E2.4 **PCA by hand.** Points $(1, 1), (2, 2), (3, 3), (4, 4)$ plus noise. Sketch the principal direction, explain variance captured, and give the 1-D projection of $(2.5, 2.7)$. When does PCA fail?

E2.5 **Pipeline.** Build a pipeline that imputes numerics with median, categoricals with mode, one-hot encodes categoricals, and robust-scales numerics. Show why fitting the scaler on train only avoids leakage with a tiny numerical example.

Chapter 3

Classification

"Classification is prediction with a finite answer." Will this email be spam? Which disease does this X-ray show? We formalise classifiers that map inputs to discrete labels.

From zero: no classifier assumed. We start from a doctor's yes/no rule, formalise the learning problem, then build three canonical classifiers from intuition to formula: decision trees, Naive Bayes, and k-nearest neighbours. Pictures precede proofs.

Learning Outcomes

After this chapter you will be able to:

(L1) formalise classification (input space, label space, hypothesis, loss);

(L2) build and prune a decision tree using entropy and information gain;

(L3) derive and apply Naive Bayes (Bayes rule + conditional independence);

(L4) apply kNN with distance metrics and discuss the curse of dimensionality;

(L5) compute confusion-matrix metrics (accuracy, precision, recall, F_1).

3.1 The Classification Problem

Intuition. Each patient is a point in feature space (fever, cough, age) coloured by a hidden label (flu or cold). Learning draws coloured regions so future points can be coloured by region. Errors happen where regions overlap.

Formal setup. Input $\mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^d$, label $y \in \mathcal{Y} = \{1, \dots, C\}$. Training data $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n \sim P(\mathbf{x}, y)$. A hypothesis $h : \mathcal{X} \to \mathcal{Y}$ incurs loss $\ell(h(\mathbf{x}), y)$, often 0-1 loss. Empirical risk $\hat{R}(h) = \frac{1}{n} \sum_i \ell(h(\mathbf{x}_i), y_i)$ approximates

true risk $R(h) = \mathbb{E}_P[\ell]$. A *decision boundary* is $\{\mathbf{x} : h(\mathbf{x}) \text{ changes}\}$. Good models balance fit and generalisation (bias-variance, Ch. 4).

3.2 Decision Trees

Intuition. A tree asks a sequence of yes/no questions (is fever > 38? is age < 10?) that partitions space into axis-aligned rectangles, each voting its majority label. Trees are interpretable but greedy and prone to overfitting.

Formal criterion. At node S with class proportions p_c, impurity:

$$\text{Entropy } H(S) = -\sum_c p_c \log_2 p_c, \qquad \text{Gini } G(S) = 1 - \sum_c p_c^2. \tag{3.1}$$

For split $S \to \{S_v\}$ on attribute A,

$$\text{Gain}(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v), \qquad \text{GainRatio} = \frac{\text{Gain}}{\text{SplitInfo}}. \tag{3.2}$$

The learner greedily picks $\arg\max_A$ Gain (ID3/C4.5) or Gini reduction (CART), recurses, and stops when pure, no attributes remain, or a pruning criterion fires. *Pruning*: pre-pruning (max depth, min samples) or post-pruning (cost-complexity: minimise $R_\alpha(T) = R(T) + \alpha|T|$).

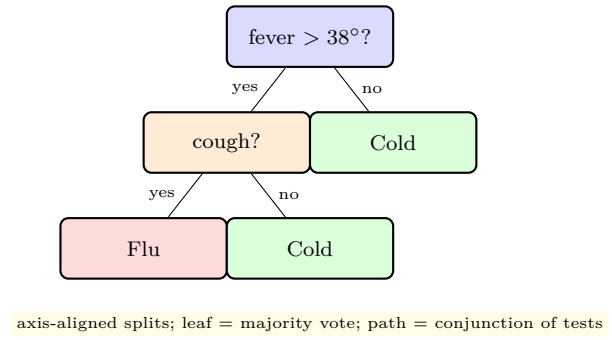

Figure 3.1: A toy decision tree for diagnosis. Each internal node is a test; traversal from root to leaf applies the conjunction along the path.

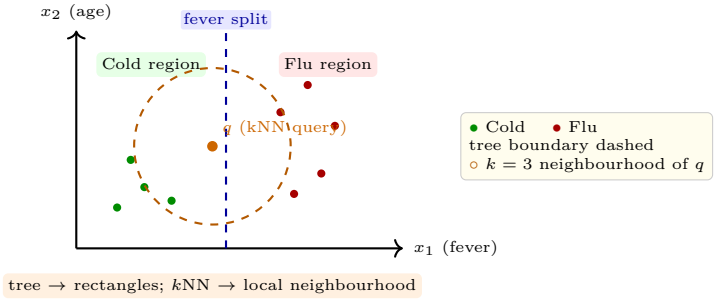

Figure 3.2: Left: geometric view. Decision trees produce axis-aligned regions; kNN classifies q by its nearest neighbours (here 2 Cold, 1 Flu → Cold for $k = 3$).

3.3 Naive Bayes

Intuition. If we know how often flu causes fever and how common flu is, we can flip the reasoning: given fever, which disease is most likely? Bayes rule does the flip; "naive" assumes symptoms are independent given the disease — often wrong but surprisingly effective.

Derivation. Posterior for class c given $\mathbf{x} = (x_1, \ldots, x_d)$:

$$P(c \mid \mathbf{x}) = \frac{P(c)\,P(\mathbf{x} \mid c)}{P(\mathbf{x})} \propto P(c) \prod_{j=1}^{d} P(x_j \mid c) \quad \text{(conditional independence)}. \tag{3.3}$$

Classify as $\hat{y} = \arg\max_c P(c) \prod_j P(x_j \mid c)$ (log-space to avoid underflow: $\sum_j \log P(x_j \mid c)$). For continuous x_j, use Gaussian $P(x_j \mid c) = \mathcal{N}(x_j; \mu_{cj}, \sigma_{cj}^2)$. Estimate $P(c)$ by frequencies, $P(x_j \mid c)$ by counts (with Laplace smoothing $+\alpha$ to handle unseen combinations).

Listing 3.1: Three classifiers side-by-side on a toy 2-D problem.

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier

X = np.array([[0,0],[1,0],[0,1],[5,5],[6,5],[5,6]])
y = np.array([0,0,0,1,1,1])  # 0=Cold, 1=Flu

tree = DecisionTreeClassifier(criterion="entropy", max_depth=3, random_state=0)
nb   = GaussianNB()
knn  = KNeighborsClassifier(n_neighbors=3, weights="uniform")

for name, clf in [("Tree",tree),("NB",nb),("3-NN",knn)]:
    clf.fit(X, y)
    print(name, clf.predict([[2,2],[5,4]]))  # expect [0,1] if separable
```

3.4 Nearest Neighbours and Evaluation Basics

kNN. Memorise all training points. For query $\mathbf{q}$, find k nearest under metric d (Euclidean d_2, Manhattan d_1, or cosine for text) and vote. $k = 1$ has low bias, high variance; larger k smooths the boundary. Needs feature scaling (Ch. 2) and suffers *curse of dimensionality*: in high d, all distances look alike, so use PCA or feature selection first. No training time, $O(nd)$ per query — accelerate with KD-trees or LSH.

Confusion-based metrics. From counts TP, TN, FP, FN:

$$\text{Acc} = \frac{TP + TN}{N}, \quad \text{Prec} = \frac{TP}{TP + FP}, \quad \text{Rec} = \frac{TP}{TP + FN}, \quad F_1 = 2\frac{\text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}. \tag{3.4}$$

Accuracy misleads on imbalanced data (99% negatives → always-negative gets 99% accuracy but zero recall). Prefer F_1 or balanced accuracy; full ROC analysis in Ch. 7.

Listing 3.2: Confusion matrix and the accuracy trap on imbalanced data.

```
from sklearn.metrics import confusion_matrix, classification_report
y_true = [1]*10 + [0]*90    # 10% positive, 90% negative
y_pred_lazy = [0]*100       # ``always negative'' model
print(confusion_matrix(y_true, y_pred_lazy))
print(classification_report(y_true, y_pred_lazy, digits=2))
# Accuracy 0.90 but recall for class 1 = 0.00 -> F1 = 0.00 !
```

3.5 A Comparative View and Practical Tips

When to prefer which. Trees excel when interpretability matters and data have mixed types and missing values; they need no scaling but overfit without pruning. Naive Bayes shines for high-dimensional sparse data (text) and tiny training sets; it is fast and calibrated when independence roughly holds. kNN has no training but expensive query, needs scaling and is sensitive to irrelevant features.

	Trees	Naive Bayes	kNN
Handles mixed types	yes	via discretisation	no (numeric)
Needs scaling	no	no	yes
Training cost	$O(nd \log n)$	$O(nd)$	$O(1)$
Query cost	$O(\text{depth})$	$O(Cd)$	$O(nd)$
Interpretability	high	moderate	low

Regularisation knobs. Trees: max depth, min samples per leaf, pruning α. Naive Bayes: smoothing α, feature selection. kNN: k, distance, weighting (uniform vs. distance-weighted), dimensionality reduction. Always tune via CV, never on test (see Ch. 7).

Common pitfall. Evaluating on training accuracy. A grown tree can memorise noise. Report stratified CV and inspect confusion, not just a single number.

3.6 Working Through Entropy by Hand

Example. At node S with 6 positives, 4 negatives: $p_+ = 0.6$, $p_- = 0.4$, so $H(S) = -0.6 \log_2 0.6 - 0.4 \log_2 0.4 \approx 0.971$ bits. Split on fever > 38 gives left $(5, 1)$ with $H_L \approx 0.65$, right $(1, 3)$ with $H_R \approx 0.81$, weighted $H_{\text{after}} = 0.6 \cdot 0.65 + 0.4 \cdot 0.81 \approx 0.714$, Gain $= 0.971 - 0.714 = 0.257$ bits.

Gini alternative. Gini for S is $1 - 0.6^2 - 0.4^2 = 0.48$; after split $G_{\text{after}} = 0.6 \cdot 0.28 + 0.4 \cdot 0.375 = 0.318$, reduction 0.162. Both criteria agree here; they can differ when splits are unbalanced — GainRatio corrects Gini's bias toward many-valued splits.

Pruning intuition. A deep tree fits training quirks; validation error first falls then rises. Cost-complexity finds the sweet spot without a separate validation set for every depth.

3.7 Worked Example: Tuning a Tree on Churn

Data. Tenure, MonthlyCharges, Contract, SupportCalls $\to$ Churn $\{0, 1\}$. A tree with max depth 2 reaches 78% CV accuracy; depth 6 reaches 84% train but 76% CV — overfitting.

Pruning path. Cost-complexity sequence for $\alpha = 0, 0.005, 0.01, 0.02$ gives nodes $31, 18, 9, 5$ and CV AUC $0.82, 0.84, 0.83, 0.79$. The best $\alpha = 0.005$ balances fit and generalisation.

Comparison. Naive Bayes on the same data (with discretised charges) scores 80% CV but calibrates well for ranking. kNN with $k = 7$ after standardisation scores 79% but is slower at prediction time. The tree wins on interpretability for business stakeholders.

Listing 3.3: Cost-complexity pruning path for decision trees.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
import numpy as np
X, y = np.random.randn(400,4), np.random.randint(0,2,400)
path = DecisionTreeClassifier(random_state=0).cost_complexity_pruning_path(X,y)
for ccp in path.ccp_alphas[::4]:
    clf = DecisionTreeClassifier(ccp_alpha=ccp, random_state=0)
    print(f"ccp={ccp:.4f}  CV={cross_val_score(clf,X,y,cv=3).mean():.3f}")
```

Reading the pruning path. The sequence of alpha values tells a story of complexity traded for stability across folds. At alpha zero the tree memorises rare contract combinations that will not recur next quarter. At alpha 0.005 the tree keeps the splits on tenure and contract that replicate across folds. Beyond that the tree becomes a stump that underfits, missing the interaction between support calls and monthly charges.

Practical model choice. The two-point gap between the tree and Naive Bayes matters less than who must act on the prediction. Retention teams prefer the tree because each path reads as a rule they can attach an offer to. Naive Bayes earns its place when calibrated churn probabilities feed a budget allocator rather than a call list. The kNN result warns that standardisation and serving latency must be weighed alongside raw accuracy.

Lessons for larger data. On millions of rows the pruning path is computed on a stratified sample first, then confirmed at scale. Categorical cardinality explodes tree depth, so high-cardinality fields are grouped before splitting. Missing contract values are routed with surrogate splits rather than dropped, preserving the deployment population.

Takeaway for deployment. Freeze the chosen alpha, retrain on all labelled data, and log the tree rules alongside the code version. Monitor leaf-level churn rates monthly, since a leaf that decays signals concept drift before global accuracy moves. Re-tune only when the monitored gap exceeds the cross-validation standard error observed during selection. That discipline turns a classroom pruning exercise into a maintainable retention model. Students should rerun the listing with different seeds to feel how stable the chosen alpha really is.

3.8 Support Vector Machines and the Kernel Trick

Margin intuition. Among all boundaries separating two classes, the SVM picks the one with the widest safety corridor: points closest to the boundary (the *support vectors*) alone determine it, so moving any other point changes nothing. Formally it minimises $\frac{1}{2}\|\mathbf{w}\|^2$ subject to $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$, giving margin width $2/\|\mathbf{w}\|$. A soft margin adds slack $\xi_i \geq 0$ with penalty $C\sum_i \xi_i$, letting a few outliers cross at a price: large C punishes violations harshly (narrow, fussy margin), small C tolerates them (wide, forgiving margin).

Kernel trick. Rings or XOR patterns are not linearly separable, so map $\mathbf{x} \mapsto \phi(\mathbf{x})$ into a higher-dimensional space where they are — but never compute ϕ explicitly, since training only needs inner products $K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\top \phi(\mathbf{z})$ (RBF $K = \exp(-\gamma\|\mathbf{x} - \mathbf{z}\|^2)$, polynomial $(1 + \mathbf{x}^\top \mathbf{z})^p$). The kernel is thus a similarity dial: RBF declares far-apart points dissimilar, and γ sets how fast similarity decays.

Worked 2D example. Positives $(2, 2), (2, 3)$, negatives $(0, 0), (1, 0)$: the boundary $x_2 = 1$ with $\mathbf{w} = (0, 1)$, $b = -1$ satisfies $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$ with equality on $(2, 2)$, $(0, 0)$, $(1, 0)$ — three support vectors — and margin width $2/\|\mathbf{w}\| = 2$. No other point matters: deleting $(2, 3)$ leaves the boundary unchanged, which is exactly why max-margin generalises.

3.9 From Labels to Numbers: Regression and Regularisation

Intuition. Regression predicts a number (price, temperature) instead of a label but reuses the same linear score $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$: classification thresholds it, regression reports it. Least squares minimises $\sum_i (y_i - f(\mathbf{x}_i))^2$ with closed form $\hat{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y}$.

Regularisation intuition. With many features, least squares memorises noise via huge cancelling weights. Ridge adds $\lambda\|\mathbf{w}\|_2^2$ (shrinks every weight smoothly); LASSO adds $\lambda\|\mathbf{w}\|_1$ (drives some weights exactly to zero, hence selects features). Larger λ means a simpler, stabler fit; tune it by cross-validation (Ch. 7), exactly like tree depth or k.

Worked fit. Points $(0, 1), (1, 3), (2, 7)$: least squares gives $y = 3x + 2/3$ (slope 3, intercept 0.67) with residuals $1/3, -2/3, 1/3$. Centred ridge with $\lambda = 1$ pulls the slope to $6/(2 + 1) = 2$, trading a little training error for stability — the same bias-variance trade-off behind pruning and large k.

Practical pick. Few thousand dense points: standardise, then try RBF-SVM against logistic regression. Sparse 10k-feature text: prefer a linear SVM (fast, strong baseline). More features than rows: reach for LASSO first, since it discards irrelevant features for you.

Common pitfall. Tuning C, γ, or λ on the test set fakes the score; tune on validation folds and report test once — Ch. 7's discipline applies to every knob in this chapter.

Listing 3.4: SVM with RBF kernel beside regularised regression.

```
from sklearn.svm import SVC
from sklearn.linear_model import Ridge, Lasso
SVC(C=1.0, kernel="rbf", gamma="scale").fit(X, y)  # max margin + kernel trick
Ridge(alpha=1.0).fit(Xr, yr)  # L2 shrinkage; Lasso(alpha=0.1) sparsifies
```

3.10 Chapter Notes

Decision trees: Quinlan ID3/C4.5, Breiman CART. Naive Bayes despite independence often rivals more complex models on text. kNN is a lazy learner — analysis via Cover-Hart (1967): asymptotically error $\leq 2\times$ Bayes optimal. For imbalance and ROC beyond accuracy, see Ch. 7.

3.11 Exercises

E3.1 **Entropy split.** A node has 6 Flu, 4 Cold. Split A gives children $(4, 1)$ and $(2, 3)$; split B gives $(3, 2)$ and $(3, 2)$. Compute H and Gain for each and choose the better split.

E3.2 **Naive Bayes by hand.** Classes $C \in \{S, H\}$ with $P(S) = 0.7$. Likelihoods $P(\text{fever} \mid S) = 0.1$, $P(\text{fever} \mid H) = 0.6$. Observe fever. Compute $P(S \mid \text{fever})$ and $P(H \mid \text{fever})$, and classify with and without Laplace smoothing if one count was zero.

E3.3 **kNN geometry.** Points $(0, 0) : 0, (1, 0) : 0, (0, 1) : 0, (3, 3) : 1$. Query $(1, 1)$. Classify for $k = 1, 3, 4$ with Euclidean distance. Explain how increasing k changes bias and variance.

E3.4 **Metrics.** Given $TP = 30, FP = 10, FN = 20, TN = 40$, compute accuracy, precision, recall, F_1. Construct a case where accuracy $= 95\%$ but recall $< 10\%$, and explain why F_1 is preferred.

E3.5 **Overfitting trees.** Explain why an unpruned tree achieves 100% training accuracy yet may generalise poorly. Propose two pruning heuristics and sketch how α in $R_\alpha(T)$ controls complexity.

Chapter 4

Ensemble Methods

"Many weak minds pooled wisely beat one strong mind." Ensembles combine many weak learners into a strong one by exploiting the bias-variance decomposition.

From zero: no ensemble assumed. We start from a single wobbly tree, explain why it wobbles (variance), then formalise bias-variance and build bagging, random forests, and boosting from intuition to algorithm.

Learning Outcomes

After this chapter you will be able to:

(L1) state the bias-variance decomposition and explain over- vs. under-fitting;

(L2) construct bagged ensembles and out-of-bag error estimates;

(L3) describe random forests (feature subsampling) and variable importance;

(L4) derive AdaBoost weight updates and gradient boosting as functional descent;

(L5) compare bagging vs. boosting and when each wins.

4.1 Bias, Variance, and Why Ensembles Help

Intuition. A single decision tree is like a strong opinion formed from one class roster: it fits quirks. A committee that averages many rosters (bagging) smooths quirks. A committee that sequentially corrects past mistakes (boosting) reduces systematic bias.

Decomposition. For regression with true f and learner $\hat{f}_{\mathcal{D}}$ on random training set $\mathcal{D}$, at point $\mathbf{x}$ with noise ε (σ^2):

$$\mathbb{E}_{\mathcal{D},\varepsilon}\Big[(y-\hat{f})^2\Big] = \underbrace{\big(\mathbb{E}_{\mathcal{D}}[\hat{f}]-f\big)^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(\hat{f}-\mathbb{E}\hat{f})^2]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible}}. \tag{4.1}$$

Simple models → high bias, low variance; complex → low bias, high variance. Bagging attacks variance; boosting attacks bias (but can increase variance if over-run). Illustration: unpruned trees have near-zero bias but large variance — ideal base learners for bagging.

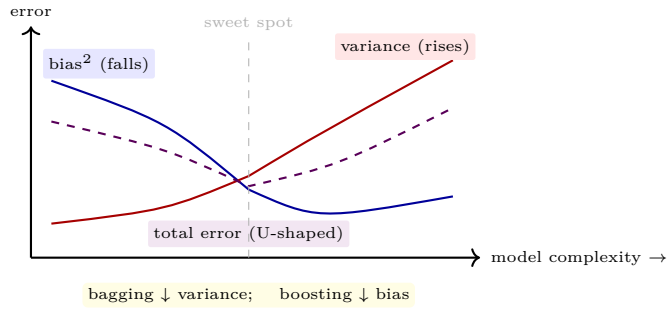

Figure 4.1: Bias-variance trade-off. The optimum balances a falling bias against a rising variance. Ensembles shift the total curve downward.

4.2 Bagging and Random Forests

Bagging (Bootstrap Aggregating). For $m = 1, \ldots, M$, draw bootstrap sample $\mathcal{D}_m$ (sample n with replacement from $\mathcal{D}$), train h_m on $\mathcal{D}_m$, aggregate: $\hat{y}$ = majority vote$(h_m(\mathbf{x}))$ (classification) or average (regression). Each point is left out of $\approx 36.8\%$ of samples ($(1 - 1/n)^n \to e^{-1}$), giving a free *out-of-bag* (OOB) estimate: predict each $\mathbf{x}_i$ using only trees that did not see it, then compute error — no separate validation needed.

Random Forest. Bagging plus *feature randomness*: at each split consider only a random subset of k features ($k \approx \sqrt{d}$ for classification, $d/3$ for regression). This decorrelates trees, further cutting variance. Variable importance: mean decrease in impurity or drop in OOB accuracy when a feature is permuted.

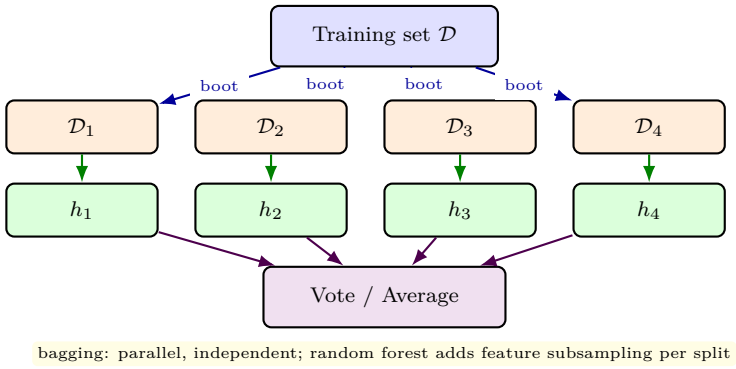

Figure 4.2: Bagging architecture. Each learner trains on an independent bootstrap view; aggregation is by voting. Random forests add random feature subsets at each split (not shown).

4.3 Boosting: AdaBoost and Gradient Boosting

AdaBoost. Initialise $w_i^{(1)} = 1/n$. For $m = 1..M$: train weak learner h_m minimising weighted error $\epsilon_m = \sum_i w_i^{(m)} \mathbf{1}[h_m(\mathbf{x}_i) \neq y_i] / \sum_i w_i^{(m)}$; compute $\alpha_m = \frac{1}{2} \ln \frac{1-\epsilon_m}{\epsilon_m}$; update $w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m y_i h_m(\mathbf{x}_i))$ (for

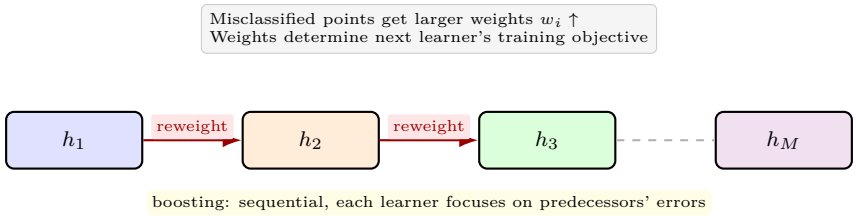

Figure 4.3: Boosting architecture. Learners are trained sequentially; misclassified points are up-weighted so the next learner attends to hard cases. Final prediction is a weighted vote.

$y_i \in \{-1, +1\}$) and renormalise. Final $H(\mathbf{x}) = \text{sign}\big(\sum_m \alpha_m h_m(\mathbf{x})\big)$. Intuition: large α_m for accurate learners; weights mushroom on points that keep being misclassified. Training error bound: $\prod_m 2\sqrt{\epsilon_m(1-\epsilon_m)} = \exp(-2\sum_m \gamma_m^2)$ where $\gamma_m = \frac{1}{2} - \epsilon_m$.

Gradient Boosting. View ensemble $F_M(\mathbf{x}) = \sum_m \nu h_m(\mathbf{x})$ as additive model built by gradient descent in function space. At step m, compute pseudo-residuals $r_i = -\partial L(y_i, F_{m-1}(\mathbf{x}_i))/\partial F_{m-1}$ and fit h_m to r_i. For squared loss, $r_i = y_i - F_{m-1}(\mathbf{x}_i)$ (ordinary residuals); for log-loss we recover LogitBoost. Shrinkage $\nu \in (0, 1]$ (learning rate) and subsampling (stochastic GB) regularise.

Listing 4.1: Bagging vs. boosting in scikit-learn: one line swaps the world.

```
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier,
    GradientBoostingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score

X, y = make_classification(n_samples=400, n_informative=8, random_state=0)

rf  = RandomForestClassifier(n_estimators=200, max_features="sqrt", oob_score=True,
    random_state=0)
ada = AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1), n_estimators=200,
    learning_rate=0.5, random_state=0)
gb  = GradientBoostingClassifier(n_estimators=200, max_depth=3, learning_rate=0.05,
    random_state=0)

for name, clf in [("RF",rf),("AdaBoost",ada),("GB",gb)]:
    scores = cross_val_score(clf, X, y, cv=5)
    print(name, f"{scores.mean():.3f} +/- {scores.std():.3f}")
print("RF OOB:", rf.fit(X,y).oob_score_)  # free validation
```

Listing 4.2: Inspecting what the forest learned: variable importance.

```
import numpy as np, matplotlib.pyplot as plt
# after fitting rf above
importances = rf.feature_importances_
idx = np.argsort(importances)[::-1]
print("Top features:", idx[:5], importances[idx[:5]])
# Permutation importance (more honest) -- see sklearn.inspection.permutation_importance
```

4.4 When to Use What

Bagging/RF: parallelisable, robust to noise/overfitting, good default; needs enough trees ($\approx$ 200–500) until OOB plateaus. Boosting: often higher accuracy on clean tabular data but sequential (harder to parallelise), sensitive to noise and outliers (they keep getting up-weighted), needs tuning of M, ν, depth. Stacking (train a meta-learner on base learners' out-of-fold predictions) can squeeze more but adds complexity. Rule: start with RF as strong baseline; try GB/XGBoost/LightGBM when you need the last few percent and data are not too noisy.

4.5 Stacking and the Practice of Winning Ensembles

Stacking. Train diverse base learners $h_1, \ldots, h_M$ with out-of-fold predictions $z_i^{(m)} = h_m^{(-fold(i))}(\mathbf{x}_i)$; then train a meta-learner g on $\{(z_i^{(1)}, \ldots, z_i^{(M)}), y_i\}$ to combine them. Linear g learns weights; nonlinear g can capture complementarity. Crucially z_i must be out-of-fold, otherwise g overfits to overconfident bases.

Why forests dominate baselines. No single hyper-parameter is critical; performance typically improves monotonically with M until OOB stabilises. Prediction is embarrassingly parallel. Feature importance from permutation is honest for auditing.

When boosting hurts. Label noise, extreme imbalance without correction, or tiny n — boosting chases noise because mislabelled points keep accumulating weight. Remedies: limit depth to 1–3, add regularisation (ν, subsample, dropout), and early-stop on validation AUC.

	Bagging / RF	Boosting
Training	parallel	sequential
Bias impact	small	large $\downarrow$
Variance impact	large $\downarrow$	can $\uparrow$ if M too large
Robust to noise	yes	less
Tuning effort	low	higher

4.6 Bias-Variance in Code: A Simulation

Idea. Fix a truth $f(x) = \sin(2\pi x)$; generate many training sets of size $n = 30$ with noise $\varepsilon \sim \mathcal{N}(0, 0.3^2)$; fit a 4-leaf tree on each; compute bias and variance at $x = 0.5$ across runs. Repeating with bagged trees shows variance drop while bias barely moves.

Listing 4.3: Simulating bias and variance for trees vs. bagged trees.

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import BaggingRegressor
np.random.seed(0)
def gen_data(n=30):
```

```
    X = np.random.rand(n,1); y = np.sin(2*np.pi*X).ravel()+np.random.randn(n)*0.3
    return X,y
Xt = np.array([[0.5]])
trials=200
preds = []
for _ in range(trials):
    X,y = gen_data(); t=DecisionTreeRegressor(max_depth=3).fit(X,y)
    preds.append(t.predict(Xt)[0])
preds=np.array(preds)
print(f"mean {preds.mean():.3f} bias {preds.mean()-np.sin(2*np.pi*0.5):.3f} var {preds.var
    ():.3f}")
```

4.7 Regularisation and Early Stopping for Boosting

Learning rate vs. number of trees. Shrinkage ν slows learning; more trees are then needed but generalisation improves. Typical $\nu = 0.05$–0.1 for GBM; $\nu = 0.01$ for XGBoost with many trees and early stopping.

Early stopping. Monitor validation AUC each iteration; stop when it has not improved for $p = 20$ rounds (patience). This automatically selects M and prevents boosting from memorising noise.

Subsampling. Stochastic gradient boosting samples a fraction $\eta = 0.5$–0.8 of rows per tree, decorrelating trees akin to bagging and speeding training. Combined with shrinkage, it is the default in modern libraries.

Listing 4.4: Early stopping for gradient boosting.

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=0)
gb = GradientBoostingClassifier(n_estimators=500, learning_rate=0.05, max_depth=3,
                                validation_fraction=0.2, n_iter_no_change=20, random_state
                                    =0)
gb.fit(X_train, y_train)
print("chosen trees:", gb.n_estimators_)
print("val AUC:", gb.score(X_val, y_val))
```

Why shrinkage works. A small learning rate forces each new tree to correct only a fraction of the residual error. No single tree can then memorise an outlier cluster, because later trees get a vote on the same points. The price is patience: hundreds of shallow trees replace dozens of aggressive ones, and training time grows. Early stopping repays that patience by halting exactly when validation AUC plateaus rather than at a guessed round.

Reading the stopping point. If validation AUC climbs then flattens for twenty rounds, the chosen ensemble sits at the bias-variance sweet spot. Stopping much earlier leaves systematic error on the table, visible as underfit on both train and validation curves. Continuing much longer memorises label noise, and validation AUC erodes

while training loss keeps falling. The patience parameter itself is a regulariser: small patience risks premature stopping on a noisy plateau.

Subsampling as a second brake. Row subsampling decorrelates consecutive trees in the same way bagging decorrelates a forest. Each tree sees a different slice of customers, so idiosyncratic rows cannot dominate every split. Column subsampling adds the random-forest benefit when a few strong predictors would otherwise starve the rest. Together shrinkage, early stopping, and subsampling form the standard trio behind most tabular competition winners.

Practical guidance. Start with depth three, learning rate 0.05, subsample 0.8, and patience twenty as a robust default. Tune depth before the learning rate, since depth controls the interaction order the ensemble can express. Log the stopped iteration as a hyperparameter, because the same data with a new seed may stop a dozen rounds away. Refit decisions should follow the validation curve, not the training loss, whenever the two disagree.

4.8 Chapter Notes

Bagging: Breiman (1996); Random Forests: Breiman (2001). AdaBoost: Freund & Schapire (1997) — the first practical boosting with exponential loss. Gradient Boosting: Friedman (2001); modern variants XGBoost, LightGBM, CatBoost dominate Kaggle tabular competitions. For theory of bias-variance, see Geman et al. (1992).

4.9 Exercises

E4.1 **Bias-variance.** Sketch bias^2, variance, and total error vs. tree depth. Mark where bagging helps most and explain why boosting can overfit if $M \to \infty$ on noisy data.

E4.2 **OOB.** With $n = 1000$, estimate the expected fraction of points not in a bootstrap sample. Explain how to compute OOB error and why it approximates k-fold CV.

E4.3 **AdaBoost weights.** Suppose $n = 4$, $w^{(1)} = [1/4, 1/4, 1/4, 1/4]$, first stump misclassifies one point with $\epsilon_1 = 0.25$. Compute α_1 and $w^{(2)}$ (unnormalised, then normalised). Which point is up-weighted most?

E4.4 **RF vs. bagged trees.** Why does feature subsampling help when a few strong predictors dominate? Give a case where random forests degrade over plain bagging.

E4.5 **Gradient boosting step.** For squared loss, show that fitting to residuals $r_i = y_i - F_{m-1}(\mathbf{x}_i)$ is a gradient step. How do learning rate ν and tree depth act as regularisers?

Chapter 5

Clustering

> *"Clustering is classification without labels."* Given points but no ground truth, we impose structure by grouping similar items and separating dissimilar ones.

From zero: no clustering assumed. We start from dots on a map, define similarity, then formalise three canonical families — partitional (k-means), hierarchical, and density-based (DBSCAN) — from picture to formula.

Learning Outcomes

After this chapter you will be able to:

(L1) define clustering, distance metrics, and the role of normalisation;

(L2) run and analyse k-means (objective, Lloyd's algorithm, elbow method);

(L3) build hierarchical clusterings (single/complete/average linkage, dendrograms);

(L4) apply DBSCAN and explain core/border/noise via density reachability;

(L5) evaluate clusters with silhouette, SSE, and external measures.

5.1 What Is Clustering?

Intuition. Scatter customers by age and spending: clumps emerge — students with low spending, professionals with high. Clustering finds such clumps without being told which clump any customer belongs to. It is *unsupervised*: only $\mathbf{x}_i$, no y_i. Applications: customer segmentation, image quantisation, gene grouping, anomaly detection.

Formal ingredients. Need (1) a representation (feature vectors), (2) a *dissimilarity* $d(\mathbf{x}, \mathbf{x}')$ (Euclidean $\|\mathbf{x} - \mathbf{x}'\|_2$, Manhattan, cosine, or Gower for mixed types), and (3) a criterion (e.g., minimise within-cluster

variance). Normalisation is mandatory: without it the feature with largest range dominates d. The number of clusters k is usually unknown and must be chosen.

5.2 Partitional: k-Means

Intuition. Place k movable centres; alternately assign each point to its nearest centre and move each centre to its cluster's mean. Points pull centres; centres claim points — until stable.

Formal objective. Given k, partition $\mathcal{C} = \{C_1, \ldots, C_k\}$ with centroids $\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \mathbf{x}$ minimises sum of squared errors:

$$\mathrm{SSE}(\mathcal{C}) = \sum_{j=1}^{k} \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|_2^2. \tag{5.1}$$

Lloyd's algorithm ("k-means") greedily descends: initialise $\boldsymbol{\mu}_j$ (random or k-means++), repeat until convergence: (E-step) assign $\mathbf{x}_i \to \arg\min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|$, (M-step) recompute $\boldsymbol{\mu}_j$ as cluster means. Complexity $O(nkd)$ per iteration; converges to a local minimum, not necessarily global — restart multiple times.

Choosing k. *Elbow*: plot SSE vs. k, look for kink where gain diminishes; *silhouette* (see §5.5) averaged over points; gap statistic; domain knowledge. k-means assumes spherical, equally sized clusters and is outlier-sensitive (one far point drags its centroid).

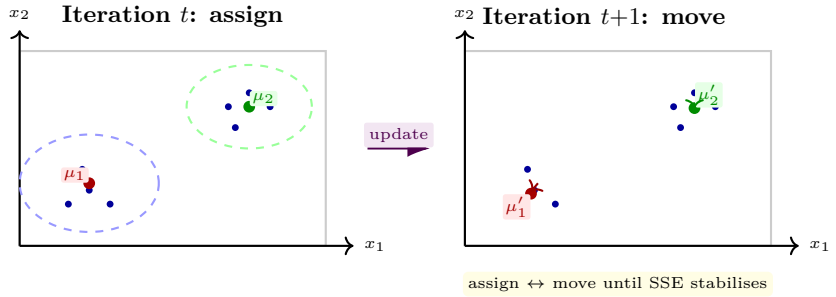

Figure 5.1: Lloyd's k-means for $k = 2$. Each iteration partitions by nearest centroid (Voronoi) then recentres. k-means++ initialises to spread centroids.

5.3 Hierarchical Clustering

Intuition. Instead of fixing k, build a tree of nested groups: start with each point alone and repeatedly merge the two closest clusters (agglomerative) or split the farthest (divisive). Cutting the resulting *dendrogram* at any height yields a clustering.

Linkage. Distance between clusters A, B depends on linkage: single $\min_{\mathbf{a} \in A, \mathbf{b} \in B} d(\mathbf{a}, \mathbf{b})$ (chaining, can produce straggly clusters), complete $\max d(\mathbf{a}, \mathbf{b})$ (compact, outlier-sensitive), average $\frac{1}{|A||B|} \sum_{\mathbf{a},\mathbf{b}} d(\mathbf{a}, \mathbf{b})$ (compromise), Ward (merge that minimises SSE increase, good default for Euclidean). Dendrogram height = linkage distance at merge; cutting at height h returns the clustering just before any merge above h.

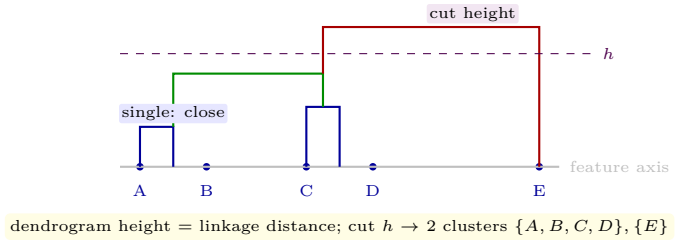

Figure 5.2: Agglomerative clustering on five points with dendrogram. Single linkage merges nearest pairs first; the dashed cut illustrates choosing k post hoc.

5.4 Density-Based: DBSCAN

Intuition. Clusters are dense blobs separated by sparse noise. DBSCAN grows clusters from dense cores outward, ignoring lonely points.

Definitions. Fix radius ε and density threshold MinPts. Point $\mathbf{p}$ is *core* if $|N_\varepsilon(\mathbf{p})| \geq$ MinPts where $N_\varepsilon(\mathbf{p}) = \{\mathbf{q} : \|\mathbf{q} - \mathbf{p}\| \leq \varepsilon\}$. Point $\mathbf{q}$ is *directly density-reachable* from $\mathbf{p}$ if $\mathbf{p}$ is core and $\mathbf{q} \in N_\varepsilon(\mathbf{p})$. *Density-reachable* is the transitive closure; a cluster is a maximal set of mutually reachable points. Points reachable from no core are *noise*. DBSCAN needs no k, finds arbitrary shapes, but is sensitive to ε and uses a global density.

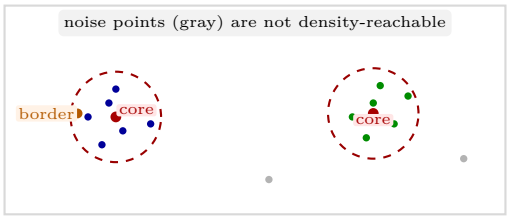

Figure 5.3: DBSCAN intuition with ε-balls. Core points seed clusters; border points lie on fringes; gray points are noise. No k is specified.

Listing 5.1: Three clusterers compared; silhouette picks the more coherent k.

```python
import numpy as np
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

X = np.array([[1,1],[1.2,1.1],[5,5],[5.2,5.1],[9,1],[9.2,0.9]])
X = StandardScaler().fit_transform(X)

km = KMeans(n_clusters=3, n_init=10, random_state=0).fit(X)
agg = AgglomerativeClustering(n_clusters=3, linkage="ward").fit(X)
db = DBSCAN(eps=0.8, min_samples=2).fit(X)

print("k-means labels:", km.labels_)
print("Agg labels   :", agg.labels_)
print("DBSCAN labels:", db.labels_)  # -1 = noise
# Silhouette (higher is better, in [-1,1])
print("silhouette k-means:", silhouette_score(X, km.labels_))
```

5.5 Evaluating Clusters

No labels, yet we need quality scores. *Internal*: SSE (compactness), silhouette for point $\mathbf{x}_i$, $s(i) = \frac{b(i)-a(i)}{\max\{a(i),b(i)\}}$ where $a(i)$ = mean intra-cluster distance, $b(i)$ = mean nearest-cluster distance; $s(i) \in [-1, 1]$, higher better; average $\bar{s}$ summarises. *External* (when labels exist for validation): Rand index, adjusted Rand, NMI. Choose distance and normalisation before comparing scores — they condition everything.

5.6 Choosing k, Scaling, and Alternatives

Elbow and gap. Run $k = 1..K_{\max}$, plot SSE or $\log \mathrm{SSE}$; the elbow where marginal gain drops is a heuristic. The *gap statistic* compares $\log \mathrm{SSE}(k)$ to its expectation under a uniform reference — more principled but costlier. Silhouette $\bar{s}(k)$ peaks at well-separated k.

Must-scale. Without standardisation, the feature with largest range dominates Euclidean distance and also SSE. Always scale after the train/test split. For mixed types, use Gower distance or encode then scale.

Beyond hard partitioning. *Gaussian mixtures* (EM) give soft assignments $P(C_j \mid \mathbf{x}_i)$ and elliptical clusters; *spectral clustering* handles non-convex shapes via graph Laplacian. DBSCAN's reachability can be extended to OPTICS for varying density, and to HDBSCAN for hierarchical density.

Listing 5.2: Elbow and silhouette for choosing k.

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import numpy as np
X = np.random.randn(120,2)
X[60:] += [4,4]
for k in [2,3,4,5]:
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    print(k, f"SSE={km.inertia_:.1f}", f"sil={silhouette_score(X, km.labels_):.2f}")
```

5.7 Distance Matters

Euclidean, Manhattan, cosine. d_2 (straight line) suits continuous features; d_1 suits grids/high-d where d_2 concentrates; cosine $1 - \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|}$ suits text (direction matters, magnitude is length). Choice changes clusters.

Mixed types. Gower distance averages per-feature contributions: for numeric, $|x_i - y_i|/\mathrm{range}_i$; for categorical, 0 if equal else 1. This lets k-prototypes or hierarchical methods handle tables with both numbers and categories.

Scaling revisited. Standardise numerics before distance; one-hot categoricals are already 0/1 but may dominate if many columns — consider weighting or embeddings.

5.8 Worked Example: Customer Segmentation

Data. 300 customers with features (Recency, Frequency, Monetary) — RFM. After log-transform and standardisation, k-means with $k = 3$ finds segments: recent frequent high spenders, dormant low spenders, and new one-time buyers.

Validation. Silhouette $\bar{s} = 0.42$ for $k = 3$ vs. 0.31 for $k = 4$, supporting three segments. Hierarchical Ward dendrogram cut at height 6.2 yields the same partition. DBSCAN with $\varepsilon = 0.6$, MinPts= 8 finds two dense cores plus 18 noise points (potential one-off outliers rather than a segment).

Action. The three segments map to actions: loyalty perks for core, re-engagement emails for dormant, onboarding offers for new. Clustering's value is not the k but the decision it enables.

Caveat. RFM segmentation drifts as behaviour changes; recluster quarterly and track segment migration to detect concept drift.

From clusters to decisions. Three RFM segments earn their keep only when each maps to a distinct marketing action with a budget. Core loyalists receive perks that reward frequency, dormant customers receive win-back offers, and new buyers receive onboarding guidance. If two clusters would receive the same treatment, merge them regardless of what the silhouette score prefers. The number of clusters is therefore a business decision informed by geometry, never dictated by it.

Why scaling decided the answer. Log-transforming monetary value stops a handful of whales from stretching the distance metric. Standardising after the transform puts recency days and frequency counts on comparable footing before any centroid moves. Without both steps the segmentation collapses to a single high-spend cluster plus everyone else, which no manager finds actionable. Students should rerun the pipeline unscaled once, purely to witness how dramatically the assignments shift.

Choosing k in practice. The elbow on SSE suggested a kink at three, and the silhouette peak at 0.42 confirmed it against four. Neither heuristic is a proof, so the Ward dendrogram cut at the same partition provides welcome corroboration. DBSCAN adds a dissenting view by labelling eighteen points as noise rather than forcing them into a segment. Those noise points deserve inspection: one-off bulk buyers behave differently from any recurring persona. A sensible workflow reports all three lenses and adopts the partition they agree on, flagging disagreements explicitly.

Stability and maintenance. Customer behaviour drifts with seasons and campaigns, so the segmentation has an expiry date. Reclustering quarterly and tracking migration between segments reveals whether the dormant pool is growing or recovering. Fix the preprocessing recipe and the random seed so quarter-to-quarter changes reflect customers rather than code. At larger scale the same logic runs on sampled data first, with full-data assignment done by nearest centroid afterwards. High-dimensional behavioural features may need dimensionality reduction before distance means anything at all.

Caveats before presenting. Clusters describe correlation, not causation: frequent buyers are not frequent because of their segment label. Pair every segment chart with its size, its defining medians, and the uncertainty from bootstrap resampling. A segment of twelve customers with a great name is an anecdote, not a strategy, and should be merged or suppressed. Document the distance, the scaling, and the chosen k alongside the slide deck so the analysis can be audited. That documentation is what separates analytics from numerology in the eyes of stakeholders. Reproduce the worked example end to end, then perturb epsilon and k to build intuition for sensitivity. The habit of validating, scaling, and questioning transfers directly to the evaluation methods of Chapter 7.

5.9 Chapter Notes

k-means: Lloyd (1957/1982); k-means++ seeding (Arthur & Vassilvitskii, 2007). Hierarchical clustering: Ward (1963). DBSCAN: Ester et al. (1996). For non-globular clusters consider spectral clustering or Gaussian mixtures (soft assignment via EM).

5.10 Exercises

E5.1 **k-means trace.** Points $(0,0),(1,0),(0,1),(10,10),(10,11),(11,10)$, $k=2$, init $\mu_1=(0,0),\mu_2=(10,10)$. Perform two Lloyd iterations, report SSE after each, and state final labels.

E5.2 **Linkage.** With points $A=0, B=1, C=5, D=6$ on a line, build single- and complete-linkage dendrograms. At what heights do merges occur? Where would you cut for $k=2$?

E5.3 **DBSCAN.** Draw six points: a dense 4-point line with $\varepsilon=1.1$, MinPts$=3$, plus two isolated points at distance 5. Identify core/border/noise and resulting clusters.

E5.4 **Normalisation trap.** Show how unnormalised Euclidean distance on features age $[0,100]$ and salary $[0,100000]$ clusters almost solely on salary. Propose a fix and its effect on SSE.

E5.5 **Silhouette.** Compute $s(i)$ for a point with $a(i)=0.5$, $b(i)=1.5$ and one with $a(i)=1.2$, $b(i)=0.8$. Interpret the sign and suggest action for the negative case.

Chapter 6

Association Analysis

"People who bought this also bought . . . " Association mining finds co-occurrence rules that drive recommendations, shelf placement, and fraud detection.

From zero: no market-basket assumed. We start from a receipt, define itemsets, support, confidence and lift, then formalise Apriori and FP-growth from intuition to complexity. Pictures precede enumeration.

Learning Outcomes

After this chapter you will be able to:

(L1) define support, confidence, and lift and interpret their trade-offs;

(L2) enumerate frequent itemsets via the Apriori principle and pruning;

(L3) build and mine an FP-tree (FP-growth) without candidate generation;

(L4) evaluate rules (lift > 1, redundancy, statistical significance);

(L5) apply association mining to a sparse transaction matrix in Python.

6.1 Rules and Interestingness

Intuition. A supermarket receipt is a basket $\{\text{bread}, \text{butter}, \text{milk}\}$. The rule $\{\text{bread}, \text{butter}\} \Rightarrow \{\text{milk}\}$ says "if a basket has bread and butter, it also tends to have milk." Usefulness depends on frequency ($\approx$ many baskets?) and reliability ($\approx$ when antecedent occurs, how often does consequent follow?) and strength beyond chance.

Formal measures. Let $\mathcal{I}$ be items, transaction $T \subseteq \mathcal{I}$, database $\mathcal{D} = \{T_1, \ldots, T_N\}$. Itemset X has *support*

$$\text{supp}(X) = \frac{|\{T \in \mathcal{D} : X \subseteq T\}|}{N} = P(X), \tag{6.1}$$

the empirical joint probability. Rule $X \Rightarrow Y$ ($X \cap Y = \emptyset$) has

$$\text{conf}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)} = P(Y \mid X), \qquad \text{lift}(X \Rightarrow Y) = \frac{\text{conf}(X \Rightarrow Y)}{\text{supp}(Y)} = \frac{P(X \cup Y)}{P(X)P(Y)}. \tag{6.2}$$

lift $= 1$ means independence; > 1 positive association; < 1 negative. Support filters rare noise; confidence measures predictive power; lift corrects for popularity of Y (a high-confidence rule can have lift ≈ 1 if Y is ubiquitous).

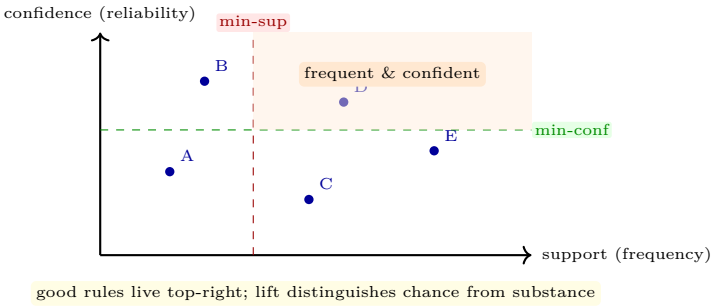

Figure 6.1: Rule space. Each point is a rule $X \Rightarrow Y$. Thresholds on support and confidence select candidates; lift then ranks by interestingness beyond baseline.

6.2 Apriori: Level-Wise Search with Pruning

Intuition. Brute-forcing all $2^{|\mathcal{I}|}$ itemsets is impossible. Apriori exploits monotonicity: if $\{a, b\}$ is infrequent, any superset $\{a, b, c\}$ is also infrequent — prune the whole subtree.

Principle and algorithm. *Apriori property*: all non-empty subsets of a frequent itemset are frequent; contrapositive gives downward closure for pruning. Algorithm (BFS level-wise): $L_1 = \{\text{frequent 1-itemsets}\}$; for $k = 2, \ldots$ generate candidates C_k by joining L_{k-1} (pairs sharing $k-2$ items), prune any $c \in C_k$ with an infrequent $(k-1)$-subset, scan $\mathcal{D}$ to count support, keep $L_k = \{c \in C_k : \text{supp}(c) \geq \text{min-sup}\}$. Stop when $L_k = \emptyset$. Then generate rules $X \Rightarrow Y$ with $X \cup Y \in \bigcup_k L_k$ and conf $\geq$ min-conf. Complexity dominated by scans and candidate blow-up on dense data.

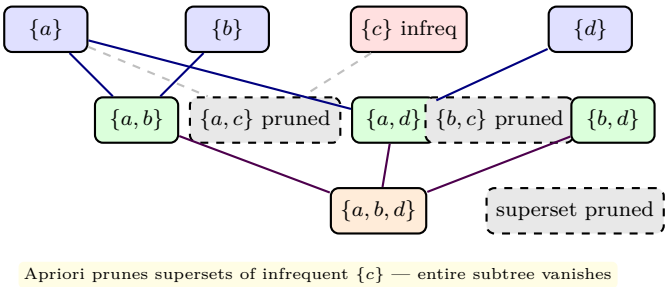

Figure 6.2: Lattice view of Apriori with min-sup pruning. Gray dashed nodes are never counted because a subset is already infrequent.

6.3 FP-Growth: Compress and Mine Without Candidates

Intuition. Instead of generating candidates, compress $\mathcal{D}$ into a prefix tree keyed by frequency order. Frequent patterns share prefixes, so the tree is compact; mining recursively extracts patterns by building conditional trees.

Steps. (1) Scan $\mathcal{D}$: count support of 1-itemsets, keep frequent, sort each transaction descending by support. (2) Insert transactions into the *FP-tree*: root $\rightarrow$ branch per item, counters along path. Header table links same items. (3) Mine: for each item i (ascending support), collect its *conditional pattern base* (prefix paths co-occurring with i), build conditional FP-tree, recurse. Each conditional tree is smaller; recursion enumerates all frequent patterns without candidate generation. FP-growth excels on dense/compressed data; worst-case still exponential (output size).

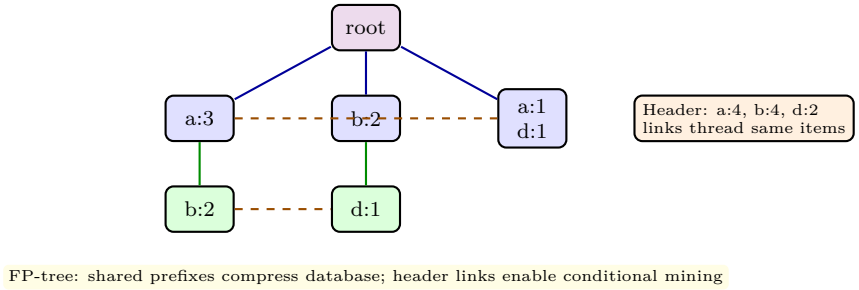

Figure 6.3: Schematic FP-tree for transactions with order $a > b > d$. Counts on nodes are supports along the prefix path; header links (dashed) chain identical items.

Listing 6.1: From transactions to rules: Apriori via mlxtend.

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# Toy transactions as one-hot DataFrame
data = [{"bread":1,"butter":1,"milk":1},
        {"bread":1,"butter":1,"milk":0},
        {"bread":0,"butter":1,"milk":1},
        {"bread":1,"butter":0,"milk":1},
        {"bread":1,"butter":1,"milk":1}]
df = pd.DataFrame(data).astype(bool)

freq = apriori(df, min_support=0.4, use_colnames=True)
rules = association_rules(freq, metric="confidence", min_threshold=0.6)
rules["lift"] = rules["confidence"] / rules["consequent support"]
print(freq.sort_values("support", ascending=False))
print(rules[["antecedents","consequents","support","confidence","lift"]])
```

Listing 6.2: Computing lift manually and filtering spurious rules.

```
# Add statistical lens: Fisher exact test for rule significance
from scipy.stats import fisher_exact
import numpy as np
# contingency for {bread,butter} => {milk}
#             milk   not-milk
#  {b,bt}      a=2      b=1
# not          c=1      d=1
```

```
a,b,c,d = 2,1,1,1
odds, p = fisher_exact([[a,b],[c,d]])
print(f"lift={a/(a+b) / ((a+c)/5):.2f}, Fisher p={p:.3f} (small p = significant)")
# In practice: correct for multiple testing (Bonferroni / FDR)
```

6.4 Judging and Using Rules

High support + high confidence is not enough: a rule {milk} ⇒ {bread} may be confident only because bread is common. Require lift > 1.2 or leverage $\text{supp}(X \cup Y) - \text{supp}(X)\text{supp}(Y) > 0$. Remove *redundant* rules where a more general antecedent has similar confidence; consider *closed* and *maximal* itemsets for compact representation. Validate causally: association $\neq$ causation (beer and nappies both peak on Fridays — common cause, not one causing the other).

6.5 Compact Representations and Sequential Patterns

Closed and maximal. An itemset X is *closed* if no proper superset has the same support; *maximal* if no superset is frequent. Closed sets preserve exact supports with far fewer patterns; maximal sets are smaller still but lose exact support. Use closed when reporting to humans.

Sequential patterns. Transactions ordered by time become sequences. Rule examples: ⟨buy phone⟩ → ⟨buy case⟩ within 7 days. Algorithms like GSP and PrefixSpan generalise Apriori to sequences where order matters and gaps are allowed.

Practical filters. Beyond lift, use *conviction* $\frac{1-\text{supp}(Y)}{1-\text{conf}}$ (∞ when rule never violates), *all-confidence* for hypercliques, and statistical tests with multiple-testing correction. Visualise top rules with grouped matrix or graph plots where nodes are items and edges are rules.

Listing 6.3: Closed itemsets intuition: filtering redundant patterns.

```
# After apriori(freq) -> keep only closed
freq["len"] = freq["itemsets"].apply(len)
# closed if no superset with same support (naive O(m2) check, OK for demo)
is_closed = []
for i, row in freq.iterrows():
    supers = freq[freq["itemsets"].apply(lambda s: row["itemsets"] < s)]
    closed = not any(supers["support"] == row["support"])
    is_closed.append(closed)
print(freq[is_closed].sort_values("support", ascending=False))
```

6.6 From Rules to Recommendations

Recommender link. Association rules power *market-basket recommenders*: if cart contains X, recommend top-k consequents Y ranked by lift $\times$ support or by confidence with a support floor. Add recency and business

constraints (margin, inventory).

Evaluation. Offline: hit-rate and mean average precision on held-out baskets. Online: A/B test click-through and revenue. A rule can be statistically significant yet commercially useless if Y is already in the cart or trivially popular.

Scaling. On 10M baskets, Apriori floods. FP-growth with partitioning (or Spark's FPGrowth) scales horizontally; sampling with error bounds (à la Toivonen) gives fast approximate frequent sets.

6.7 Chapter Notes

Agrawal, Imielinski & Swami (1993) introduced association rules; Apriori is Agrawal & Srikant (1994); FP-growth is Han et al. (2000) — now the default for sparse large data. For sequential patterns see PrefixSpan; for recommender context see Ch. 8.

6.8 Exercises

E6.1 **Measures.** Transactions: 100 total; $\{a\}$ in 40, $\{b\}$ in 50, $\{a, b\}$ in 20. Compute support, confidence, and lift for $\{a\} \Rightarrow \{b\}$ and $\{b\} \Rightarrow \{a\}$. Is the association positive?

E6.2 **Apriori trace.** Items $\{1, 2, 3, 4\}$, transactions $\{\{1, 2, 3\}, \{1, 2\}, \{1, 3\}, \{2, 3, 4\}\}$, min-sup $= 0.5$ (≥ 2 transactions). List L_1, L_2, L_3 and pruned candidates at each level.

E6.3 **FP-tree.** Build an FP-tree for transactions $\{\{a, b\}, \{b, c\}, \{a, b, c\}, \{a, c\}\}$ with order by global frequency. Show header table and the conditional base for item c.

E6.4 **Lift vs. confidence.** Give a dataset where a rule has confidence 0.9 but lift ≈ 1. Explain why lift matters for recommendation and name one alternative measure (leverage, conviction).

E6.5 **Multiple testing.** Why does mining thousands of rules inflate false positives? Propose a correction (Bonferroni or FDR) and discuss its trade-off.

Chapter 7

Evaluation and Model Selection

"Without honest evaluation, every model looks brilliant." We turn from building models to judging them rigorously and choosing among them.

From zero: no evaluation assumed. We start from a lucky coin-flip classifier, formalise train/test discipline, then build cross-validation, ROC analysis, and imbalance handling from picture to formula.

Learning Outcomes

After this chapter you will be able to:

(L1) implement hold-out, k-fold, stratified, and leave-one-out validation;

(L2) compute and interpret confusion-matrix, ROC, AUC, and PR curves;

(L3) diagnose overfitting (learning curves, validation curves);

(L4) handle class imbalance (resampling, SMOTE, cost-sensitive, thresholding);

(L5) compare models with confidence intervals and paired tests.

7.1 Honest Estimation: Hold-Out and Cross-Validation

Intuition. Training error is optimism. Like an exam, you need unseen questions. The *test set* is that unseen exam, touched only once at the end. *Cross-validation* (CV) is the mock-exam schedule that estimates how stable your score is.

Formal setup. Split $\mathcal{D}$ into train $\mathcal{D}_{\text{tr}}$ and test $\mathcal{D}_{\text{te}}$ (e.g., 70/30, stratified to preserve class ratios). Report metrics on $\mathcal{D}_{\text{te}}$ only. For model selection/tuning, add a *validation* split or use k-fold CV: partition $\mathcal{D}_{\text{tr}}$ into k folds $F_1, \ldots, F_k$; for $i = 1..k$ train on $\bigcup_{j \neq i} F_j$, test on F_i, average scores $\bar{s} = \frac{1}{k}\sum_i s_i$ with std $\sigma/\sqrt{k}$. Leave-one-out (LOO) is $k = n$ (unbiased but high variance and cost). Stratified k-fold preserves class proportions each

fold — crucial for imbalanced data. *Golden rule*: all preprocessing that learns from data (imputation, scaling, selection) must be fitted inside each fold, not before splitting, otherwise leakage.

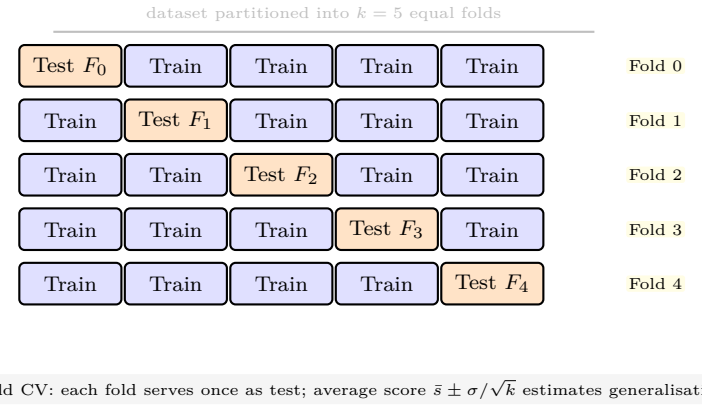

Figure 7.1: k-fold cross-validation. Blue = training in that iteration, orange = held-out test. The mean across folds is the CV estimate; its standard error gauges stability.

7.2 Confusion, ROC, and Precision-Recall

Intuition. A threshold turns a score into a yes/no. Sliding the threshold trades false alarms against missed cases. ROC visualises this trade-off; precision-recall does when positives are rare.

Definitions. For binary $y \in \{0, 1\}$ and score $s(\mathbf{x}) \in \mathbb{R}$, predict $\hat{y} = \mathbf{1}[s \geq \tau]$. Rates:

$$\mathrm{TPR}(\tau) = \frac{TP}{TP + FN} \text{ (recall)}, \qquad \mathrm{FPR}(\tau) = \frac{FP}{FP + TN}. \tag{7.1}$$

ROC curve: parametric plot $(\mathrm{FPR}(\tau), \mathrm{TPR}(\tau))$ as τ sweeps $\mathbb{R}$; diagonal = random, top-left = perfect. *AUC* $= \int_0^1 \mathrm{TPR}\, d\mathrm{FPR} = P(s(\mathbf{x}^+) > s(\mathbf{x}^-))$ — probability a random positive outranks a random negative. PR curve plots (recall, precision); AUC-PR is preferred for heavy imbalance where ROC looks optimistic.

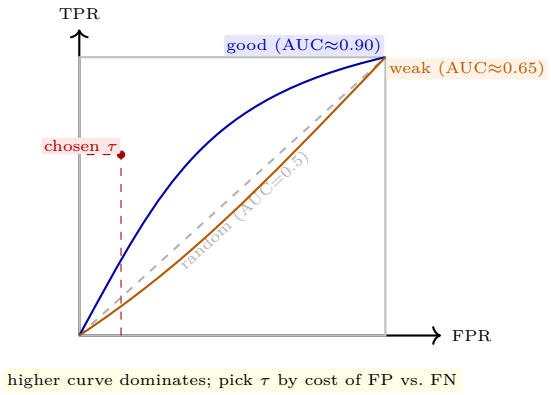

Figure 7.2: ROC space. Sweeping threshold τ from ∞ (origin) to $-\infty$ (top-right) traces each curve. AUC summarises ranking quality.

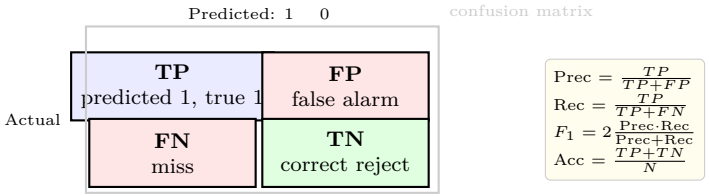

Figure 7.3: Binary confusion matrix and derived metrics. Precision penalises false alarms; recall penalises misses; F_1 is their harmonic mean.

7.3 Imbalance and Calibration

Problem. With 1% fraud, a "no-fraud" classifier gets 99% accuracy yet is useless. Metrics must reflect minority-class performance: prefer PR-AUC, F_1, balanced accuracy, or expected cost $C_{FP}FP + C_{FN}FN$.

Remedies. Data level: random oversampling (duplicate minorities), undersampling (drop majorities), SMOTE (synthesise points along segments between nearest minority neighbours). Algorithm level: class-weighted loss ($w_c \propto 1/N_c$), cost-sensitive thresholds (choose τ minimising expected cost). Always evaluate on the original distribution after resampling only the training folds — test must stay realistic.

Listing 7.1: Honest CV: preprocessing inside each fold; imbalance handled correctly.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import make_scorer, f1_score, roc_auc_score

# Pipeline ensures scaling is fit per fold (no leakage)
pipe = Pipeline([("scale", StandardScaler()),
                 ("clf", LogisticRegression(class_weight="balanced", max_iter=500))])

from sklearn.datasets import make_classification
X, y = make_classification(n_samples=600, weights=[0.9,0.1], random_state=0)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
f1 = cross_val_score(pipe, X, y, cv=cv, scoring=make_scorer(f1_score)).mean()
auc = cross_val_score(pipe, X, y, cv=cv, scoring="roc_auc").mean()
print(f"F1={f1:.3f}  AUC={auc:.3f}")
```

Listing 7.2: ROC, PR, and threshold choice by expected cost.

```
from sklearn.metrics import roc_curve, precision_recall_curve, auc
import numpy as np
pipe.fit(X, y)
proba = pipe.predict_proba(X)[:,1]
fpr, tpr, thr = roc_curve(y, proba)
prec, rec, _ = precision_recall_curve(y, proba)
print("ROC AUC:", auc(fpr, tpr))
# Cost-sensitive threshold: minimise C_FP*FP + C_FN*FN
C_FP, C_FN = 1, 10
best = min(zip(thr, fpr*sum(y==0)*C_FP + (1-tpr)*sum(y==1)*C_FN), key=lambda x: x[1])
print("cost-optimal threshold ~", best[0])
```

7.4 Beyond a Single Number: Comparison and Learning Curves

Learning curves plot train and validation error vs. training size: converging with a gap → high variance (need more data or simpler model); both high → high bias (need more features or more expressive model). *Validation curves* sweep a hyper-parameter (e.g., tree depth, k). To claim A beats B, use paired tests on CV folds (paired t or Wilcoxon) or McNemar on the test set, and report confidence intervals, not just means. For multiple models, correct for multiplicity (Bonferroni).

7.5 Learning Curves and Statistical Comparison

Learning curves. Plot $\text{error}_{\text{train}}(n)$ and $\text{error}_{\text{val}}(n)$ vs. n. High-bias: both high and converged; high-variance: validation above training with a gap. The shape tells whether to collect more data (variance) or enrich the model (bias).

Nested CV. When tuning hyper-parameters, the CV score of the best configuration is optimistic. *Nested CV* outer-loops estimate generalisation: outer folds evaluate the whole tuning procedure that inner CV performs. Report outer mean, not the inner best.

Comparing classifiers. For paired folds, a paired t-test on differences $d_i = s_i^A - s_i^B$ tests $H_0 : \mu_d = 0$ via $t = \bar{d}/(s_d/\sqrt{k})$. McNemar on the test set uses disagreements: $\chi^2 = (|b - c| - 1)^2/(b + c)$ where $b = A$ right B wrong, $c =$ opposite. Always pair this with effect size and domain cost — a statistically significant 0.2% gain may be practically irrelevant.

Listing 7.3: Learning curve and nested CV sketch.

```
from sklearn.model_selection import learning_curve, GridSearchCV, cross_val_score
from sklearn.tree import DecisionTreeClassifier
import numpy as np
X, y = np.random.randn(200,5), np.random.randint(0,2,200)
train_sizes, train_scores, val_scores = learning_curve(
    DecisionTreeClassifier(), X, y, cv=5, train_sizes=np.linspace(0.2,1.0,5))
print("train", train_scores.mean(axis=1))
print("val  ", val_scores.mean(axis=1))
# nested CV
inner = GridSearchCV(DecisionTreeClassifier(), {"max_depth":[2,4,None]}, cv=3)
outer = cross_val_score(inner, X, y, cv=5)
print("nested CV", outer.mean(), outer.std())
```

7.6 Calibration and Cost Curves

Calibration. A model that says "70% chance of churn" should be right 70% of the time. Reliability diagrams bin predictions and plot mean predicted vs. observed frequency; Platt scaling or isotonic regression recalibrates.

Cost curves. When misclassification costs are asymmetric (e.g., missing disease is 10× worse than a false alarm), ROC's iso-cost lines slope $\frac{P(-)C_{FP}}{P(+)C_{FN}}$. The operating point is where the ROC meets the appropriate cost line — not necessarily where accuracy is maximal.

Reporting. Always report mean ± standard error across folds, and state what was tuned on what data. A result without variance is anecdotal.

7.7 Case Study: From Accuracy to Business Impact

Scenario. Fraud detection with 0.8% positives. Model A: accuracy 99.2%, recall 12%; Model B: accuracy 97.1%, recall 68%, precision 18%. Accuracy favours A, but expected cost with $C_{FN} = 50 \cdot C_{FP}$ favours B by a factor of four.

Threshold tuning. Moving threshold from 0.5 to 0.22 (optimal under the cost ratio via the validation PR curve) lifts recall 45% → 68% at a precision cost 25% → 18%, yet net savings improve because catching fraud dominates.

Reporting. Stakeholders need ±SE intervals: "AUC 0.91 ± 0.02 (5-fold CV), PR-AUC 0.43 ± 0.04" and a cost curve showing savings vs. threshold, not a single F1. This aligns evaluation (Ch. 7) with Business Understanding (Ch. 1).

Why accuracy lied. With 0.8 percent fraud, a classifier that never flags anything scores 99.2 percent accuracy and catches nothing. The cost matrix exposes the lie: one missed fraud costs fifty times one false alarm, so recall dominates the business objective. Model B sacrifices headline accuracy to multiply recall more than fivefold, and expected cost falls by a factor of four. Any evaluation that ignores costs in this setting optimises the wrong function, however carefully cross-validated.

Thresholds as policy. Moving the decision threshold from 0.5 to 0.22 converts the probability ranking into an action tuned to the cost ratio. The validation precision-recall curve shows exactly what that move buys: recall rises from 45 to 68 percent while precision eases to 18 percent. Review teams then staff to the resulting alert volume rather than to an arbitrary 0.5 cutoff inherited from balanced problems. Recalibration after threshold moves keeps reported probabilities honest for downstream triage queues.

Uncertainty belongs in the report. Point estimates of AUC hide fold-to-fold variation that stakeholders need for resourcing. Reporting AUC 0.91 with a standard error of 0.02 across five folds communicates how

repeatable the ranking really is. The wider interval on PR-AUC reflects the scarcity of positives and warns against overclaiming on the minority class. Nested cross-validation belongs wherever thresholds or costs were tuned, so the outer loop measures the whole procedure. Leakage checks complete the story: scaling and sampling inside folds, never on the pooled data before splitting.

Calibration and cost curves. Reliability diagrams reveal whether a quoted 70 percent fraud risk materialises 70 percent of the time. Platt scaling or isotonic regression repairs miscalibration before thresholds derived from costs are applied. Cost curves then display expected loss against operating point, letting finance choose the threshold they will fund. That plot replaces arguments about F1 with a shared view of money against risk appetite.

From case study to checklist. Every future evaluation can reuse the same sequence: define costs, tune thresholds on validation, report with intervals. Preserve the random seeds, fold assignments, and cost assumptions so an auditor can replay the numbers exactly. Monitor live precision and recall weekly, because fraudsters adapt and yesterday's optimal threshold decays. When live cost drifts beyond the validation interval, retrain and re-tune rather than nudging the threshold by hand. Document the chosen operating point next to the confusion matrix so accuracy is never quoted without its cost context. That discipline connects the statistical machinery of this chapter back to the business understanding of Chapter 1. Students should replicate the threshold sweep on any imbalanced dataset to feel the precision-recall trade directly. Try cost ratios of ten and a hundred to see how violently the optimal point moves under identical ROC curves. The lesson generalises: metrics are choices about consequences, not neutral summaries of correctness. A model is finished when its evaluation justifies a funded decision, not when a single number looks large.

7.8 Chapter Notes

CV theory: Stone (1974) and Geisser. ROC introduced for radar (Peterson & Birdsall, 1953); AUC as ranking statistic in Hanley & McNeil (1982). SMOTE: Chawla et al. (2002). Rule: if you touch the test set more than once, it is no longer a test set — use nested CV for tuning.

7.9 Exercises

E7.1 **Leakage.** A pipeline standardises the whole dataset, then does CV. Explain the leakage with a toy numeric example and fix the code by moving scaling inside the fold.

E7.2 **ROC by hand.** Scores $[0.1, 0.4, 0.35, 0.8]$ with labels $[0, 0, 1, 1]$. Enumerate thresholds $\infty, 0.8, 0.4, 0.35, 0.1$, compute TPR/FPR, sketch ROC, and estimate AUC.

E7.3 **Imbalance.** With 95% negatives, a model predicts all negatives. Compute accuracy, precision, recall, F_1. Propose two fixes (one data-level, one threshold-level) and say which metric each optimises.

E7.4 **Learning curves.** Sketch curves for high-bias vs. high-variance models as n grows. How would you diagnose which regime you are in from the gap between train and validation error?

E7.5 **Model comparison.** Two models give CV accuracies $[0.81, 0.83, 0.80, 0.82, 0.84]$ and $[0.78, 0.80, 0.79, 0.81, 0.80]$. Perform a paired t-test at $\alpha = 0.05$ (or argue qualitatively) whether the difference is significant and discuss practical vs. statistical significance.

Chapter 8

Advanced Topics: Text, Time Series, and Big Data

"Most of the world's data is text, ticks, and streams." We extend mining beyond tables to sequences, language, and data that never fits on one machine.

From zero: no advanced data assumed. We start from an SMS, a stock ticker, and a log stream, reformulate each as a mining problem, then formalise text representation, time-series motifs, and distributed computation. Pictures precede formalism.

Learning Outcomes

After this chapter you will be able to:

(L1) transform text into numeric vectors (bag-of-words, TF-IDF, embeddings);

(L2) describe time-series components and similarity (trend, seasonality, DTW);

(L3) outline MapReduce/Spark execution and streaming sketches;

(L4) explain scalability bottlenecks (shuffle, skew, velocity) and mitigations;

(L5) scope an end-to-end project combining text/time-series at scale with ethical checks.

8.1 Mining Text

Intuition. "I love this phone" and "I hate this phone" differ by one word but opposite sentiment. Text must be turned into numbers that preserve discriminating patterns while discarding noise.

Bag-of-words pipeline. Tokenisation (split on whitespace/punctuation, lower-case, handle emojis) → normalisation (stemming "running"→"run", lemmatisation) → stop-word removal → vocabulary V → vector

$\mathbf{x} \in \mathbb{R}^{|V|}$ with term frequency $\mathrm{tf}(t, d)$. Down-weight common words via inverse document frequency:

$$\mathrm{idf}(t) = \log \frac{N}{\mathrm{df}(t)}, \qquad \text{tf-idf}(t, d) = \mathrm{tf}(t, d) \times \mathrm{idf}(t), \tag{8.1}$$

where $\mathrm{df}(t)$ counts documents containing t. TF-IDF up-weights rare but document-specific terms. n-grams (pairs/triples) recover some order; embeddings (Word2Vec, contextual transformers) map words to dense vectors where cosine approximates meaning.

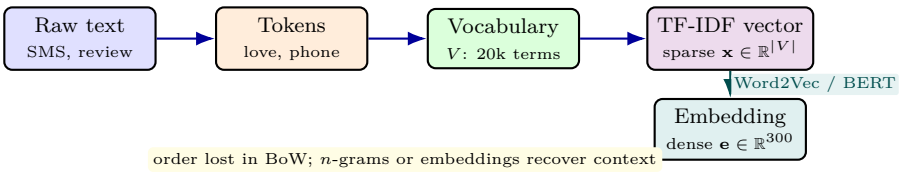

Figure 8.1: Text-to-vector pipeline. Bag-of-words treats documents as multisets of terms; TF-IDF reweights by informativeness; embeddings compress to dense semantics.

8.2 Time Series and Sequences

Intuition. A heart-rate trace has trend (rising), seasonality (daily cycle), and surprise (arrhythmia). Mining asks: what is expected, what deviates, and which traces look similar?

Formal view. Time series $y_1, \ldots, y_T$. Decomposition $y_t = T_t + S_t + R_t$ (trend + seasonal + residual) via moving averages or STL. Stationarity (μ, σ^2 constant) is assumed by many models; differencing $\nabla y_t = y_t - y_{t-1}$ helps. Similarity: Euclidean requires equal length and alignment; *Dynamic Time Warping* (DTW) allows warping: $\mathrm{DTW}(Q, C) = \min_{\text{warp path } W} \sum_{(i,j)\in W} d(q_i, c_j)$ with monotonicity and boundary constraints (DP in $O(T^2)$, often pruned with LB-Keogh bounds). Motifs/discords: repeated/anomalous subsequences; forecasting (ARIMA, exponential smoothing) vs. classification (similarity-based or feature-based).

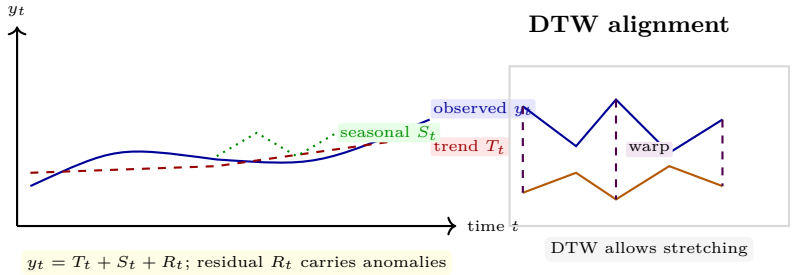

Figure 8.2: Left: additive decomposition. Right: DTW aligns sequences by warping time (dashed) so similar shapes match despite speed/phase shifts.

8.3 Big Data and Streaming

Intuition. Data that does not fit on one machine, or never stops arriving, needs a different discipline: move computation to data, and answer from summaries/sketches.

MapReduce / Spark. *Map*: $(\text{key}, \text{value}) \mapsto \text{list}(k', v')$; *shuffle/sort* by k'; *Reduce*: $(k', \text{list}(v')) \mapsto \text{result}$. Fault-tolerant via replication and recomputation. Spark generalises to DAGs of RDDs/DataFrames with in-memory caching; DataFrame API and Catalyst optimiser made it practical. Key costs: *shuffle* (network) and *skew* (one key dominates — mitigate by salting).

Streams. Unbounded $x_1, x_2, \ldots$ with one pass and sublinear memory. Sketches: Count-Min for frequency, HyperLogLog for cardinality, reservoir sampling for uniform sample, sliding windows with exponential decay. For mining on streams: incremental k-means, Hoeffding trees, and approximate FP-growth variants.

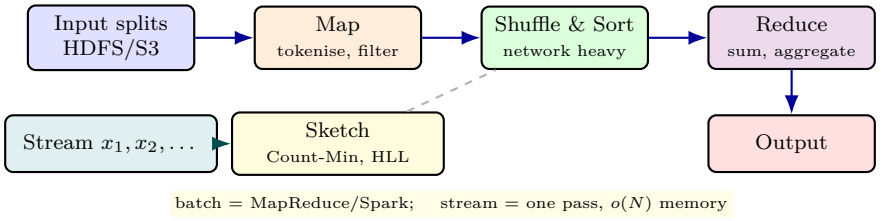

Figure 8.3: Batch vs. stream. MapReduce/Spark scales by splitting and shuffling; streaming sacrifices exactness for bounded memory and real-time latency.

Listing 8.1: Text + TF-IDF + classifier: a minimal sentiment pipeline.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

docs = ["I love this phone", "I hate this phone", "love love great", "hate terrible awful"]
y = [1,0,1,0]  # 1=positive, 0=negative
pipe = Pipeline([("tfidf", TfidfVectorizer(stop_words="english", ngram_range=(1,2))),
                 ("clf", LogisticRegression())])
pipe.fit(docs, y)
print(pipe.predict(["love this great phone", "terrible hate"]))
print(pipe.named_steps["tfidf"].vocabulary_)
```

Listing 8.2: MapReduce word count locally; then think distributed.

```
from collections import Counter
import itertools

# Map: emit (word,1) per document
def mapper(doc): return [(w.lower(),1) for w in doc.split()]

# Shuffle+Reduce: group by key and sum
docs = ["hello world", "hello data mining", "world of data"]
mapped = list(itertools.chain.from_iterable(mapper(d) for d in docs))
counts = Counter()
for k,v in mapped: counts[k]+=v
print(counts)  # Counter({'hello':2,'world':2,'data':2,'mining':1,'of':1})
# On Spark: sc.textFile("hdfs://...").flatMap(...).map(lambda w:(w,1)).reduceByKey(lambda a,b:a+b)
```

8.4 Putting It Together: An End-to-End Sketch

A production pipeline stitches chapters: CRISP-DM scoping (Ch. 1) → preprocessing and pipelines with no leakage (Ch. 2) → modelling (Ch. 3–6) → honest evaluation under imbalance (Ch. 7) → text/time-series features and scalable execution (this chapter). Add *MLOps*: versioned data, reproducible pipelines, monitored drift, and ethical review (fairness, privacy, consent) at every stage. When data or the world shifts, the cycle repeats.

8.5 Recommenders and Graph Mining at Scale

Collaborative filtering. User-item matrix R_{ui} (ratings or implicit clicks) is sparse. Latent-factor models factorise $R \approx PQ^\top$ where $\mathbf{p}_u, \mathbf{q}_i \in \mathbb{R}^k$ ($k \approx 20$–200) and $\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i$. Optimise $\sum_{(u,i)\in\mathcal{K}} (r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i)^2 + \lambda(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2)$ via SGD/ALS (Spark MLlib does this distributively). Neighbourhood models use item-item cosine on columns of R.

Graph mining. Many big datasets are graphs (social, web, transaction). Tasks: community detection (Louvain — modularity), PageRank (random-walk stationary), and pattern counting on edges. All motivate distributed graph engines (GraphX, Pregel) where computation moves to vertices.

Feature store and monitoring. At scale, features are served from a *feature store* (offline for training, online for serving) to prevent skew. Monitor data drift (PSI, KL) and concept drift (performance decay); automate retraining triggers but gate on ethical review — especially when feedback loops recycle model predictions as future training data.

Listing 8.3: ALS-style recommender intuition in pure Python (tiny demo).

```
import numpy as np
R = np.array([[5,3,0,1],[4,0,0,1],[1,1,0,5]], dtype=float)
mask = R>0
# trivial rank-2 ALS step (one iteration)
k=2
P = np.random.randn(3,k); Q = np.random.randn(4,k)
# fix Q, solve P row-wise (closed form for this demo: gradient step)
lr=0.01
for _ in range(200):
    pred = P @ Q.T
    err = (R - pred)*mask
    P += lr * (err @ Q)
    Q += lr * (err.T @ P)
print(np.round(P@Q.T,2))
```

8.6 Ethics and MLOps for Advanced Data

Bias in text/time-series. Word embeddings encode historical bias ("doctor" closer to "he" than "she"); time-series models trained on past regimes fail when regimes shift (COVID, market crash). Audit embeddings

and stress-test temporal models on out-of-period data.

Privacy. Text may contain PII; time series may be re-identifiable (heart trace). Techniques: differential privacy (add calibrated noise), k-anonymity for quasi-identifiers, and federated learning that keeps raw data on device.

MLOps loop. Log data lineage, feature definitions, and model versions; monitor drift (PSI > 0.2 triggers review); canary-deploy new models; require human approval for high-stakes actions. Ethics is not a final chapter — it closes the CRISP-DM loop back to Business Understanding.

8.7 Chapter Notes

Text: Manning, Raghavan & Schütze, *Introduction to Information Retrieval* (2008) for TF-IDF; Mikolov et al. (2013) for Word2Vec. Time series: Hyndman & Athanasopoulos, *Forecasting* (2021); DTW: Berndt & Clifford (1994). Big data: Dean & Ghemawat (MapReduce, 2004); Zaharia et al. (Spark, 2012). Streaming sketches: Cormode & Muthukrishnan (Count-Min).

8.8 Exercises

E8.1 **TF-IDF.** Corpus of 3 docs, term "data" appears in 2 docs with tf $2, 1, 0$. Compute idf and tf-idf for doc 1. Why does a term in every document get zero idf?

E8.2 **DTW.** Sequences $Q = [1, 2, 3]$, $C = [1, 2, 2, 3]$. Compute DTW distance with $d(q_i, c_j) = |q_i - c_j|$ by filling the DP matrix and tracing the optimal warp path.

E8.3 **MapReduce.** Design map and reduce for counting co-occurring item pairs within baskets. Where does skew arise and how would salting help?

E8.4 **Decomposition.** Sketch a series with linear trend + yearly seasonality + noise. Explain how differencing removes trend and why STL is preferred over a single moving average for seasonal extraction.

E8.5 **Ethics at scale.** A streaming loan-approval model retrains nightly on recent data. Describe a failure mode where feedback loops amplify bias, and propose a monitoring metric and an intervention.